

BÀI GIẢNG: COMPLEX VARIABLES

TS. Đỗ Sính

Nội dung bài học sẽ tập trung giải quyết các mục tiêu: Phân biệt được các kiểu dữ liệu Enum, Array, Mapping và Struct trong ngôn ngữ lập trình Solidity. Hiểu được, giải thích được và cho các ví dụ minh họa các kiểu dữ liệu trên.

Kết thúc nội dung này, sinh viên trả lời được các câu hỏi sau:

Câu hỏi 1: Phân biệt Enum, Array, Mapping, và Struct trong Solidity?

Câu hỏi 2: Cho ví dụ về việc sử dụng Enum và giải thích?

Câu hỏi 3: Cho ví dụ về việc sử dụng Array và giải thích?

Câu hỏi 4: Cho ví dụ về việc sử dụng Mapping và giải thích?

Câu hỏi 5: Cho ví dụ về việc sử dụng Struct và giải thích?

1. Array

Array có thể có kích thước cố định trong thời gian biên dịch hoặc chúng có thể có kích thước động.

Kiểu của mảng có kích thước cố định k và kiểu phần tử T được viết là $T[k]$ và kiểu của mảng có kích thước động là $T[]$.

Mảng là một cấu trúc dữ liệu, nó lưu trữ một tập hợp tuần tự có kích thước cố định của các phần tử cùng kiểu.

Mảng được sử dụng để lưu trữ một tập hợp dữ liệu, nhưng thường hữu ích hơn nếu coi một mảng như một tập hợp các biến cùng kiểu.

Thay vì khai báo các biến riêng lẻ, chẳng hạn như `number0`, `number1`, ... và `number99`, bạn khai báo một biến mảng chẳng hạn như `số` và sử dụng `số[0]`, `số[1]` và ..., `số[99]` để biểu diễn các biến riêng lẻ. Một phần tử cụ thể trong một mảng được truy cập bởi một chỉ mục.

Trong Solidity, một mảng là danh sách có thứ tự các mục được lập chỉ mục bằng số, bắt đầu từ 0.

Mảng được sử dụng để lưu trữ một tập hợp dữ liệu.

Trong Solidity, một mảng có thể có kích thước cố định theo thời gian biên dịch hoặc kích thước động. Đối với mảng lưu trữ, nó cũng có thể có các loại phần tử khác nhau. Trong trường hợp mảng bộ nhớ, kiểu phần tử không thể được ánh xạ và trong trường hợp nó được sử dụng làm tham số hàm thì kiểu phần tử phải là kiểu ABI.

Tất cả các mảng bao gồm các vị trí bộ nhớ liền kề. Địa chỉ thấp nhất tương ứng với phần tử đầu tiên và địa chỉ cao nhất cho phần tử cuối cùng.

Để khai báo một mảng có kích thước cố định trong Solidity, người lập trình chỉ định kiểu của các phần tử và số phần tử theo yêu cầu của một mảng như sau: `arrayName [arraySize]` (đây được gọi là mảng một chiều).

Kích thước mảng phải là một hằng số nguyên lớn hơn 0 và kiểu có thể là bất kỳ kiểu dữ liệu Solidity hợp lệ nào. Ví dụ, để khai báo một mảng 10 phần tử được gọi là số dư kiểu uint, hãy sử dụng câu lệnh này: `uint balance[10];`

2. Enum

Enum là một cách để tạo một kiểu do người dùng xác định trong Solidity. Chúng có thể chuyển đổi rõ ràng và từ tất cả các số nguyên nhưng không cho phép chuyển đổi ngầm định.

Enum có tối đa không quá 256 giá trị

Enums hạn chế một biến chỉ có một trong một số giá trị được xác định trước. Các giá trị trong danh sách được liệt kê này được gọi là enum.

Chuyển đổi rõ ràng từ kiểm tra số nguyên trong thời gian chạy rằng giá trị nằm bên trong phạm vi của enum và gây ra một xác nhận không thành công nếu không. Enums cần ít nhất một thành viên.

Enum trong Solidity là viết tắt của Enumerable. Chúng là kiểu do người dùng xác định chứa các tên có thể đọc được của con người cho một tập hợp các hằng số, được gọi là thành viên(member).

Enum cần ít nhất một thành viên.

Bạn không thể sử dụng số (dương hoặc âm) hoặc boolean (đúng hoặc sai ở dạng chữ thường) làm thành viên cho một enum. Tuy nhiên, True và False (Viết hoa) được chấp nhận.

Enum hữu ích khi bạn có một biến có thể nhận một trong một loạt các giá trị được xác định trước (hoặc từ một tập hợp nhỏ các giá trị có thể có).

Enum có thể được sử dụng để tạo các loại tùy chỉnh với một số hữu hạn là " giá trị không đổi".

Bạn không cần phải kết thúc khai báo enum bằng dấu chấm phẩy. Trình biên dịch sử dụng dấu chấm phẩy; và dấu ngoặc nhọn {} để xác định phạm vi của mã mà nó được áp dụng.

Các tùy chọn được biểu diễn bằng các giá trị số nguyên không dấu tiếp theo bắt đầu từ 0, theo thứ tự chúng được xác định.

Biểu diễn dữ liệu giống như đối với enums trong C: Các tùy chọn được biểu diễn bằng số nguyên không dấu tiếp theo giá trị bắt đầu từ 0.

3. Mapping

Bạn khai báo các kiểu ánh xạ với ánh xạ cú pháp mapping (`_KeyType => _ValueType`). `_KeyType` có thể là bất kỳ loại sơ cấp nào. Điều này có nghĩa là nó có thể là bất kỳ kiểu giá trị tích hợp nào cộng với byte và string. `_ValueType` có thể là bất kỳ kiểu nào, bao gồm cả các ánh xạ.

Ánh xạ chỉ có thể có loại storage và thường được sử dụng cho các biến trạng thái.

Ánh xạ có thể được đánh dấu công khai. Solidity tự động tạo getter cho nó.

Bạn có thể coi Mapping là hàm băm, hầu như được khởi tạo sao cho mọi khóa có thể tồn tại và được ánh xạ tới một giá trị có biểu diễn byte.

Ánh xạ không có độ dài hoặc khái niệm về khóa hoặc giá trị được đặt và do đó không thể bị xóa nếu không có thông tin bổ sung liên quan đến các khóa được chỉ định.

Các ánh xạ chỉ có thể có 1 vị trí storage (lưu trữ) dữ liệu và được phép cho các biến trạng thái, như các kiểu tham chiếu lưu trữ trong các hàm hoặc như các tham số cho các hàm thư viện.

Chúng không thể được sử dụng làm tham số hoặc tham số trả về của các hàm.

4. Struct

Được sử dụng để biểu diễn một bản ghi.

Struct trong solidity chỉ là một kiểu tùy chỉnh mà bạn có thể xác định. Bạn xác định Struct với tên và các thuộc tính liên quan bên trong nó.

Các kiểu Struct có thể được sử dụng bên trong các ánh xạ và mảng dưới dạng ValueType và bản thân chúng có thể chứa ánh xạ và mảng.

Một Struct có thể chứa một mảng có kích thước động thuộc kiểu riêng của nó.

Trong Solidity, một ánh xạ được tham chiếu đến một bảng băm, bao gồm các loại khóa và các cặp loại giá trị.

Từ khóa struct xác định một kiểu dữ liệu mới, có nhiều hơn một thành viên.

Không thể cho một Struct chứa một thành viên của kiểu riêng của nó, mặc dù bản thân Struct có thể là kiểu giá trị của một thành viên ánh xạ hoặc nó có thể chứa một mảng có kích thước động thuộc loại của nó. Hạn chế này là cần thiết, vì kích thước của struct phải là hữu hạn.

Trong tất cả các hàm, kiểu Struct được gán cho một biến cục bộ có lưu trữ vị trí dữ liệu.

Có thể sử dụng chức năng xóa tích hợp Solidity trên một biến kiểu Struct để đặt lại tất cả các thành viên của nó. Điều này sẽ hoạt động giống như nếu biến sẽ được khai báo là một cấu trúc mà không gán bất kỳ giá trị nào cho các thành viên của nó.

Để xác định một Structs, bạn phải sử dụng từ khóa struct. Từ khóa struct xác định một kiểu dữ liệu mới, có nhiều hơn một thành viên.

Các kiểu struct có thể được sử dụng bên trong các mapping và array và bản thân chúng có thể chứa các mapping và array.

Không thể cho một cấu trúc chứa một thành viên có kiểu riêng của nó, mặc dù bản thân cấu trúc có thể là kiểu giá trị của một thành viên ánh xạ hoặc nó có thể chứa một mảng có kích thước động cùng kiểu của nó. Hạn chế này là cần thiết vì kích thước của cấu trúc phải là hữu hạn.

Kiểu cấu trúc được gán cho một biến cục bộ có vị trí dữ liệu lưu trữ. Điều này không sao chép cấu trúc mà chỉ lưu trữ một tham chiếu để các phép gán cho các thành viên của biến cục bộ thực sự ghi vào trạng thái.

LAB 7:

Trước hết, ta tạo 1 folder và trong folder đó tạo 1 file với tên là StartStopUpdate.sol hoặc bạn có thể đặt tên tùy ý.

Để thực hiện một Smart Contract. Ta cần khai báo phiên bản đang sử dụng của Solidity đang ở phiên bản bao nhiêu.

Ta dùng “pragma”:

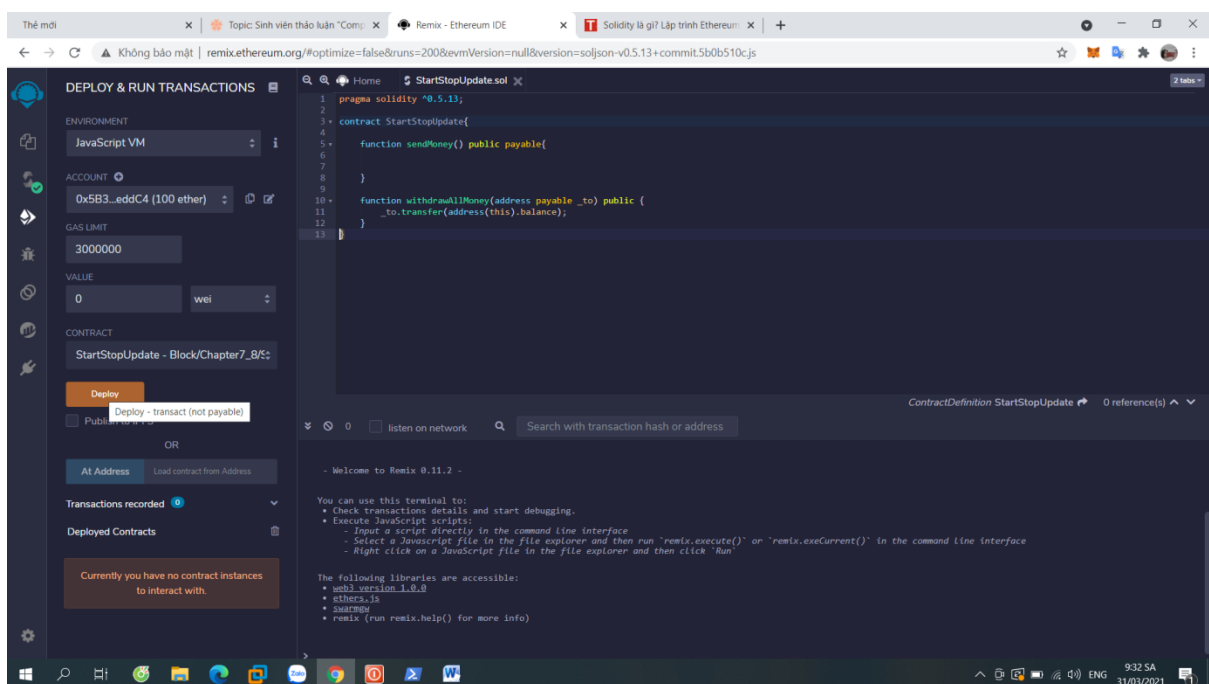
```
pragma solidity ^0.5.13;
```

Tiếp theo ta tạo một Smart Contract với tên StartStopUpdate{}

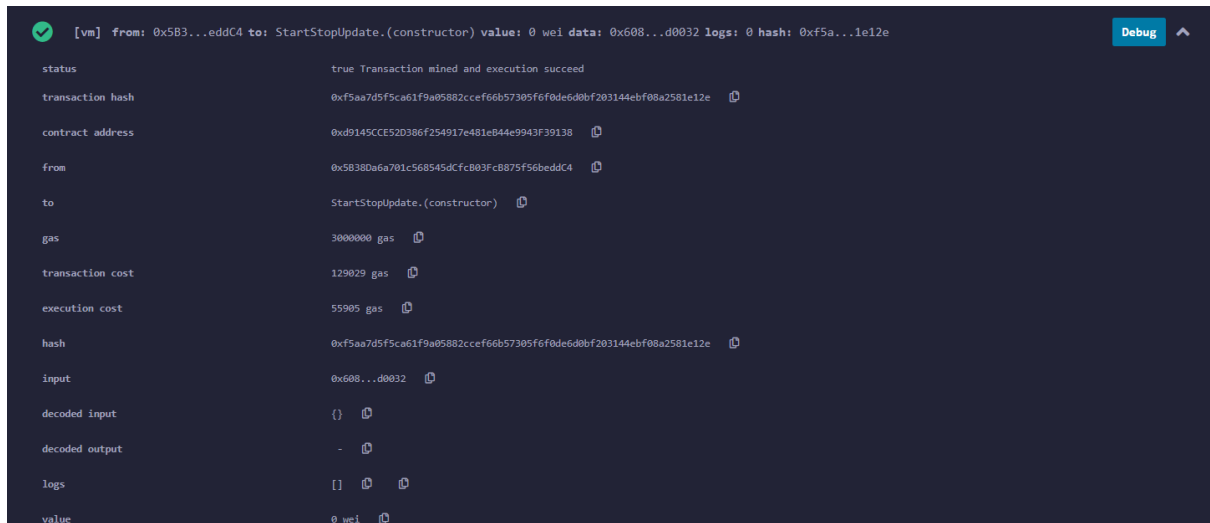
Tạo 1 function sendMoney(): Chức năng này có thể nhận Ether, đây là một chức năng có thể thanh toán với việc sử dụng modifier payable.

Tạo 1 function withdrawAllMoney(): chức năng này sẽ tự động rút tất cả các khoản tiền có sẵn được lưu trữ tại địa chỉ của Smart Contract thành biến trong địa chỉ đối số hàm như đã cho. Ở đây, cụ thể là gửi tới địa chỉ “_to”.

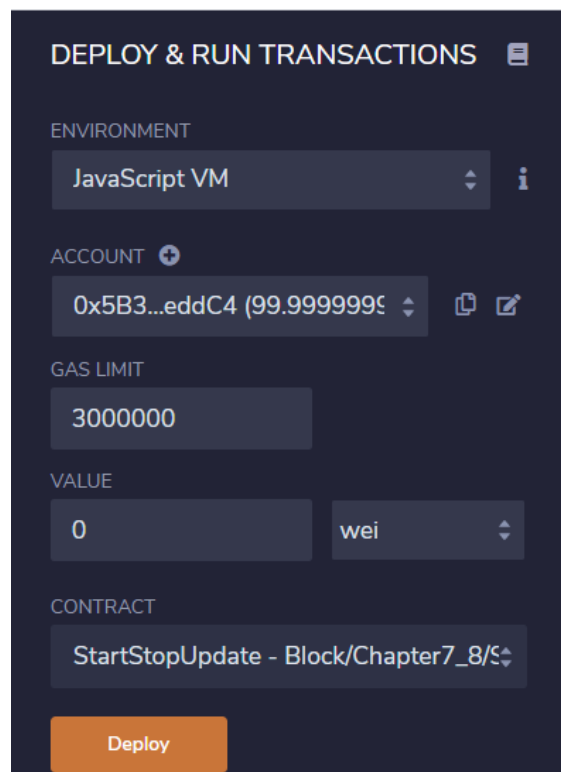
Bây giờ, ta tiến hành triển khai Smart Contract.



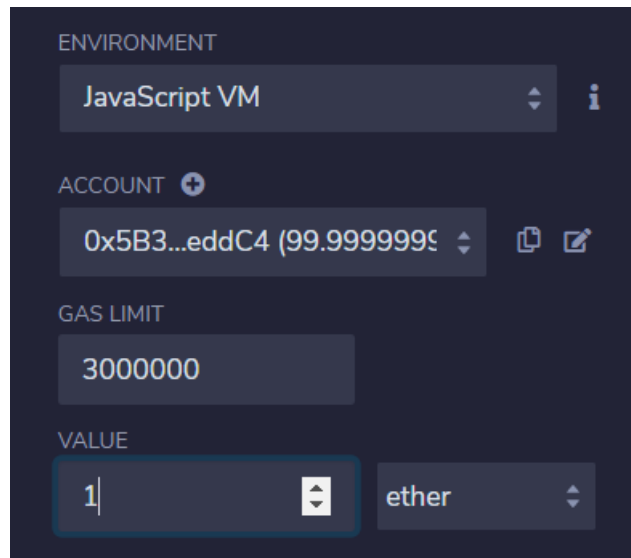
Khi đó, giao dịch sẽ được gửi đi và hiển thị thông báo trong bảng biểu của Remix:



Lúc này, ta nhận thấy địa chỉ ví khi deploy sẽ giảm đi 1Ether.



Bây giờ, ta thực hiện gửi đi 1Ether. Ta nhập vào giá trị Value là 1 và đơn vị là Ether.



ENVIRONMENT

JavaScript VM

ACCOUNT +

0x5B3...eddC4 (99.99999999)

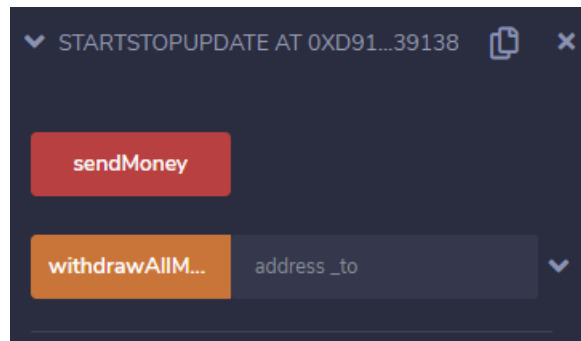
GAS LIMIT

3000000

VALUE

1 ether

Ta xuống xuống và nhấn vào sendMoney để thực hiện gửi ether.

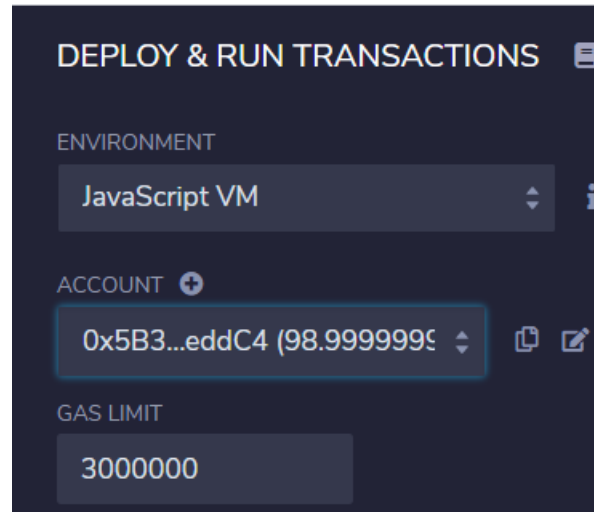


STARTSTOPUPDATE AT 0XD91...39138

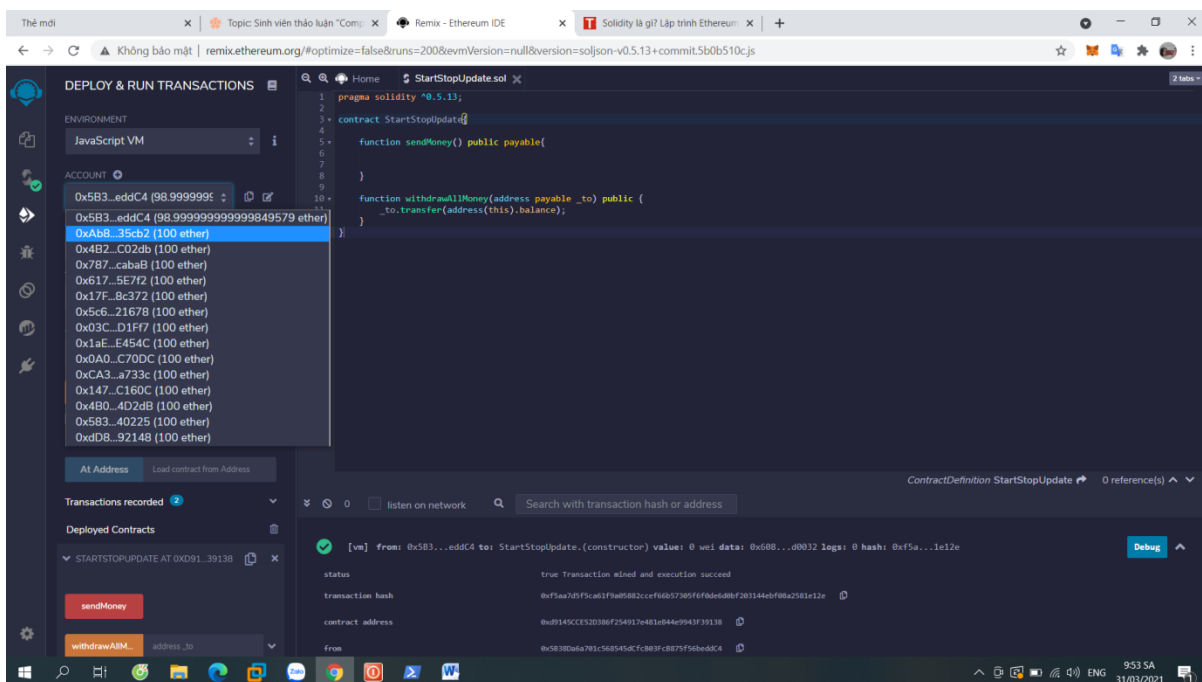
sendMoney

withdrawAllM... address _to

Và kết quả là 1 ether đã được gửi đi.



Để chuyển tiền cho 1 địa chỉ khác, ta chọn 1 địa chỉ bất kì trong danh sách tài khoản.

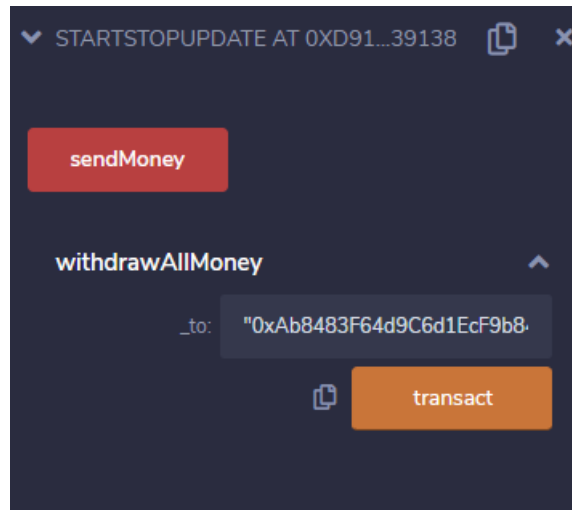


Ở đây, mình chọn địa chỉ thứ 2 từ trên xuống.

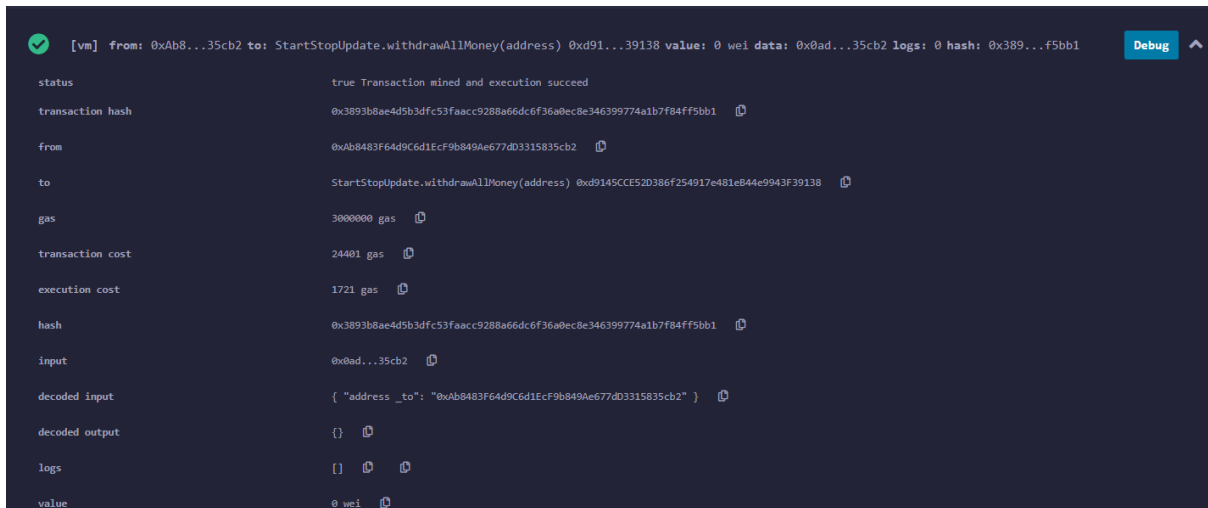
Nhấn chọn nút để copy địa chỉ



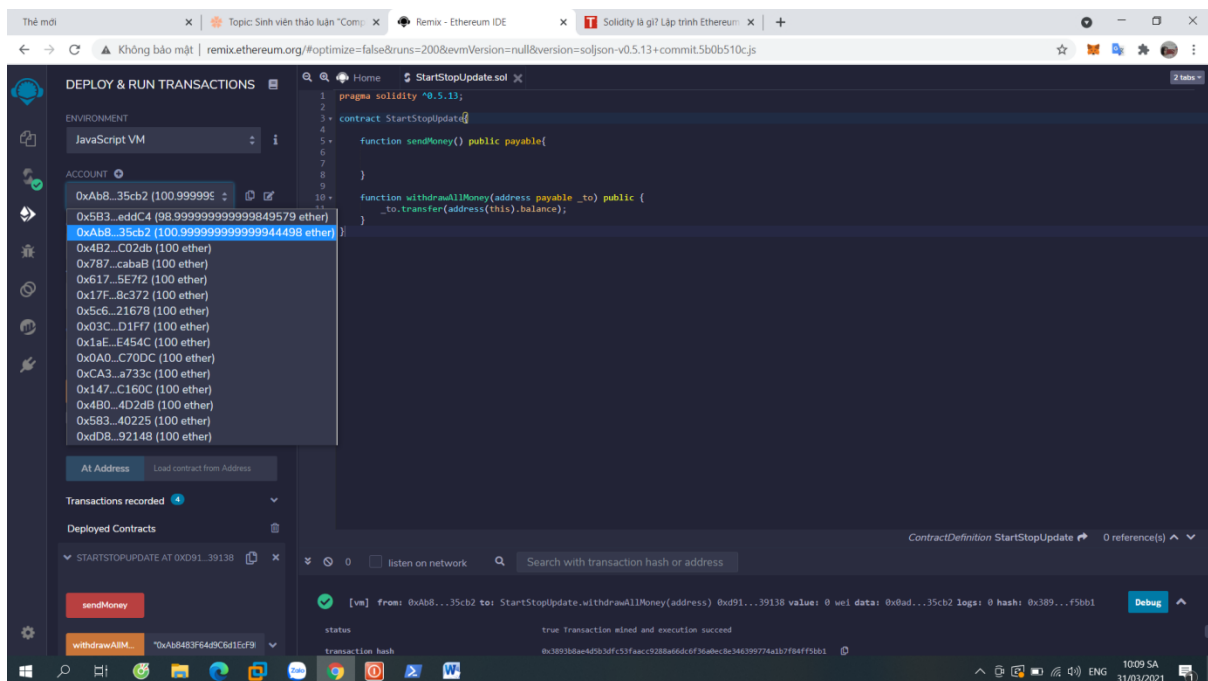
Dán địa chỉ đã sao chép vào trường nhập “withdrawAllMoney”



Nhấp vào nút “transact” và giao dịch sẽ thực hiện lúc này sẽ hiển thị thông báo.



Và ta nhận thấy, tài khoản thứ 2 trong danh sách tài khoản đã nhiều hơn 100ether.



Để bảo vệ an toàn 1 Smart Contract, chúng ta sẽ thực hiện như sau:

Chúng ta sẽ đặt một biến lưu trữ thành địa chỉ đã triển khai Smart Contract. Sau đó, ta sẽ `require()` người tương tác với rút tiền cũng giống như người đã triển khai Smart Contract.

```
pragma solidity ^0.5.13;

contract StartStopUpdate{

    address owner;

    constructor() public{
        owner = msg.sender;
    }

    function sendMoney() public payable{

    }

    function withdrawAllMoney(address payable _to) public {
        require(msg.sender == owner, "You are not the owner");
        _to.transfer(address(this).balance);
    }
}
```

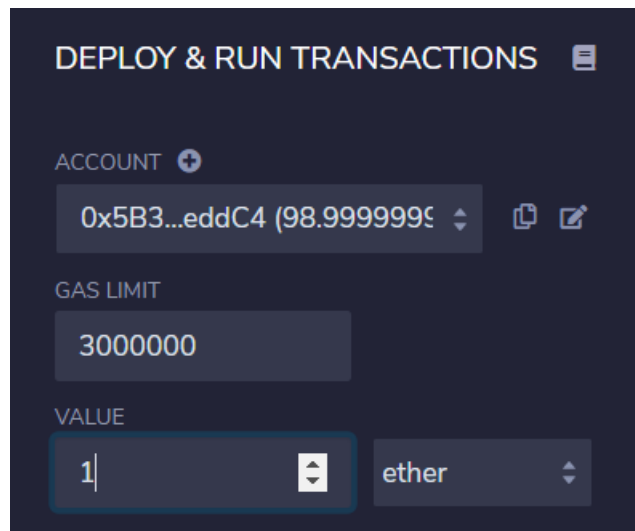
+ `constructor ()`: là một hàm đặc biệt chỉ được gọi một lần trong quá trình triển khai hợp đồng. Nó vẫn có các đối tượng toàn cầu giống như trong bất kỳ giao dịch nào khác.

+ Vì vậy, trong `msg.sender` là địa chỉ của người đã triển khai Smart Contract.

+ `require(msg.sender == owner, "You are not the owner")`: đây là cách để kích hoạt lỗi trong Solidity. Nếu yêu cầu đánh giá là sai, nó sẽ dừng giao dịch, khôi phục mọi thay đổi đã thực hiện và hiện thông báo lỗi dưới dạng String.

Giờ chúng ta thực hiện triển khai một Smart Contract

Để gửi 1 ether đến Smart Contract ta nhập vào trường Value là 1



The screenshot shows a dark-themed interface for deploying and running transactions. At the top, the title 'DEPLOY & RUN TRANSACTIONS' is displayed with a hamburger menu icon. Below this, the 'ACCOUNT' section shows a selected account with the address '0x5B3...eddC4' and a balance of '(98.99999999 ether)'. The 'GAS LIMIT' is set to '3000000'. The 'VALUE' section has a text input field containing '1' and a dropdown menu set to 'ether'. The input field for '1' is highlighted with a red border.

DEPLOY & RUN TRANSACTIONS

ACCOUNT 

0x5B3...eddC4 (98.99999999 ether)  

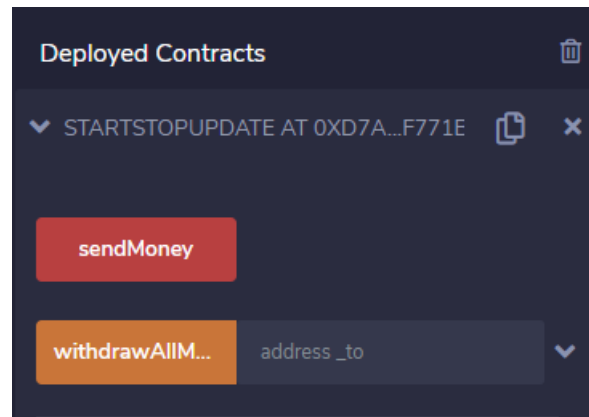
GAS LIMIT

3000000

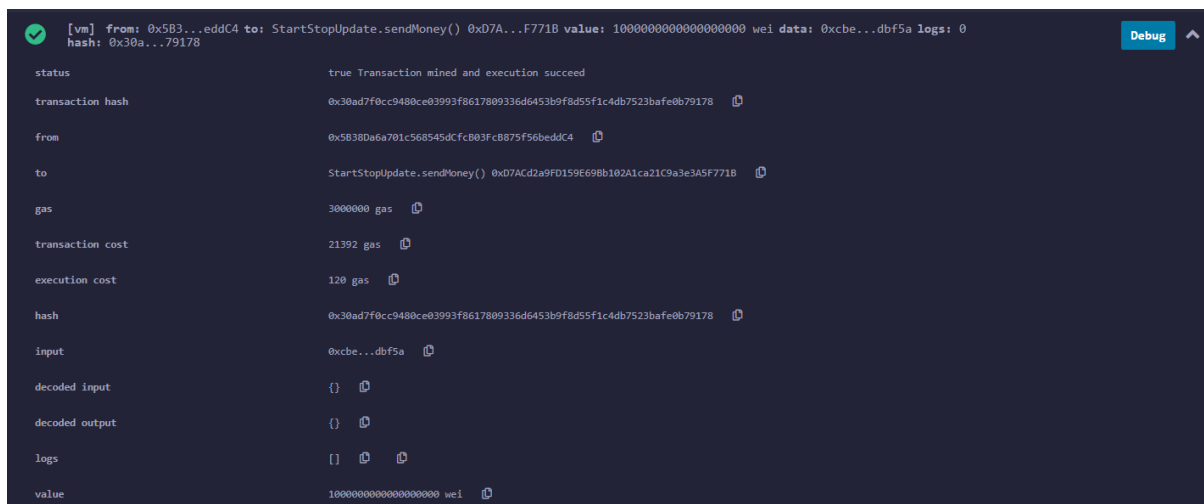
VALUE

1  ether 

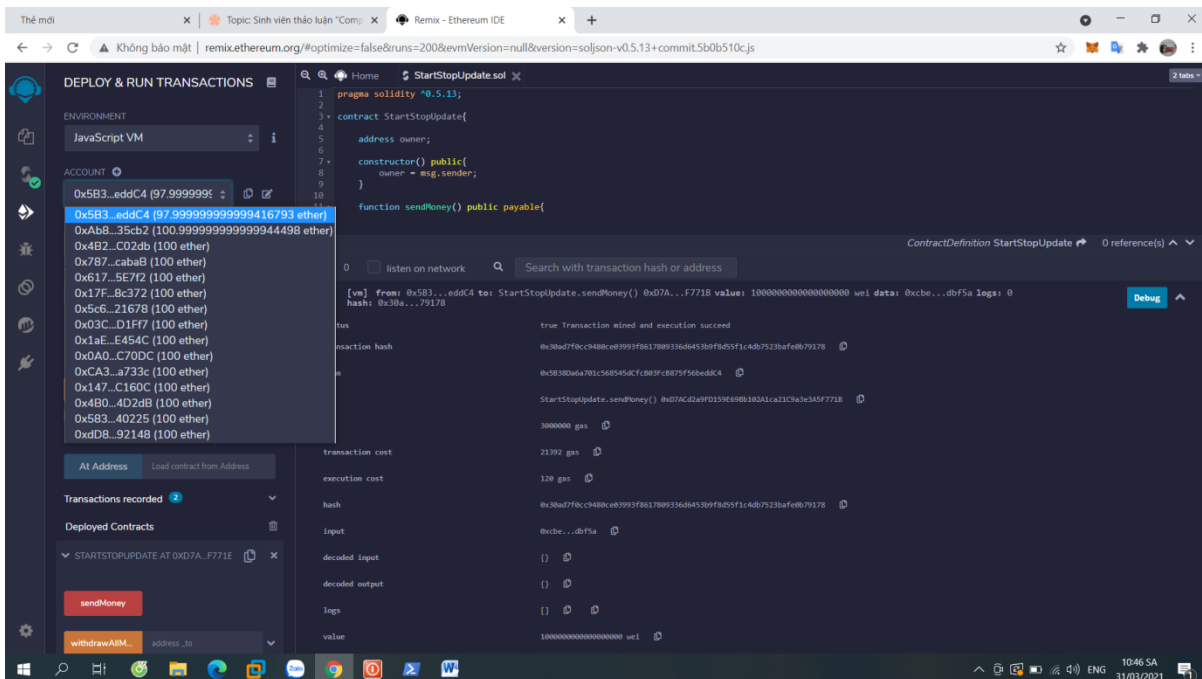
Sau đó kéo xuống và nhấp vào nút “sendMoney” để thực hiện gửi ether.



Khi đó, Remix sẽ hiển thị thông báo giao dịch thành công.

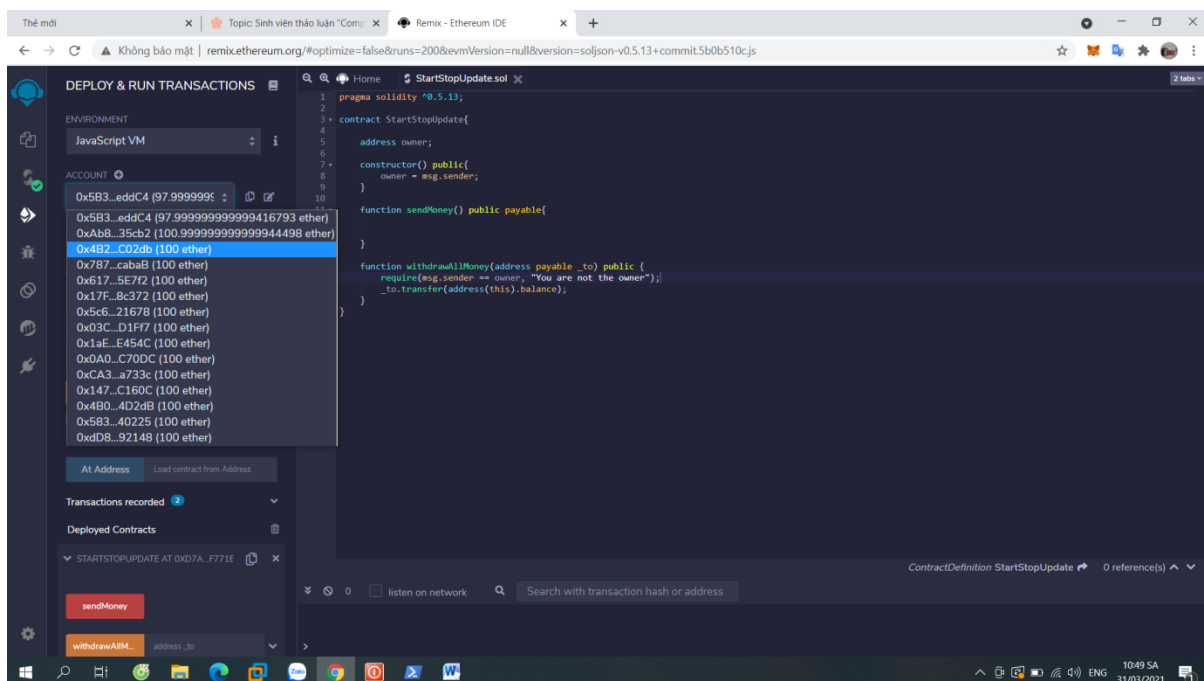


Kết quả nhận lại đó là tài khoản sẽ bị trừ 1 ether.

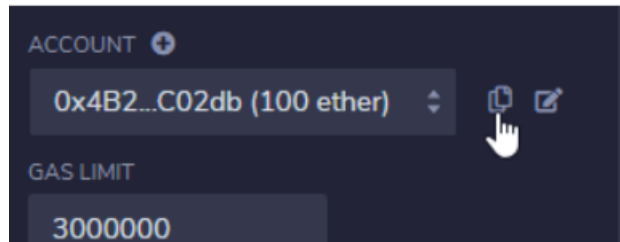


Tiếp theo ta dùng 1 tài khoản khác để rút tiền.

Ta chọn 1 địa chỉ trong danh sách tài khoản từ trên xuống. Mình chọn tài khoản thứ 3.



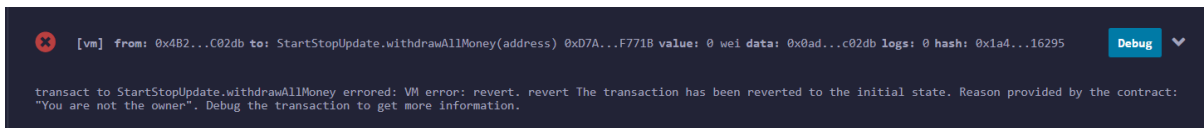
Và ta tiến hành sao chép địa chỉ đó.



Dán địa chỉ vào trường nhập “withdrawAllMoney”



Nhấp vào nút “withdrawAllMoney” để cùng xem điều gì xảy ra nhé.



Thông báo lỗi sẽ hiển thị,

Như vậy, một quy tắc truy cập rất đơn giản là từ chối quyền truy cập của bất kỳ ai ngoại trừ người đã triển khai Smart Contract.

Nếu muốn gỡ lỗi như trên, ta quay lại tài khoản ban đầu và thực hiện rút tiền.

Paused trong Smart Contract:

Bây giờ chúng ta cập nhật Smart Contract và thêm một biến “bool”.

```
pragma solidity ^0.5.13;

contract StartStopUpdate{

    address public owner;
    bool public paused;

    constructor() public{
        owner = msg.sender;
    }

    function sendMoney() public payable{

    }

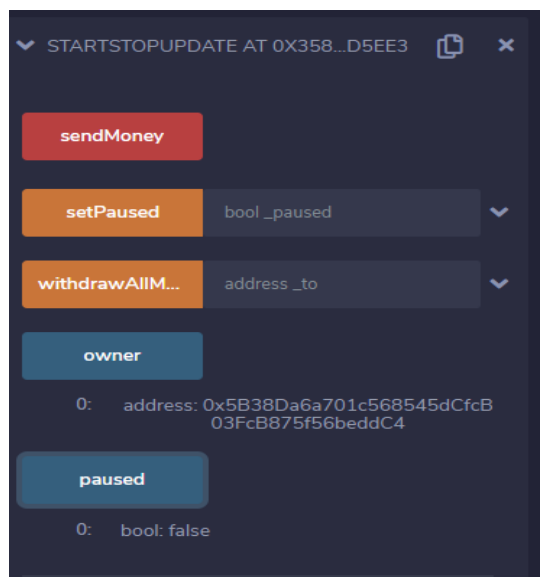
    function setPaused(bool _paused) public{
        require(msg.sender == owner, "You are not the owner");
        paused = _paused;
    }

    function withdrawAllMoney(address payable _to) public {
        require(msg.sender == owner, "You are not the owner");
        require(!paused, "Contract is paused!");
        _to.transfer(address(this).balance);
    }
}
```

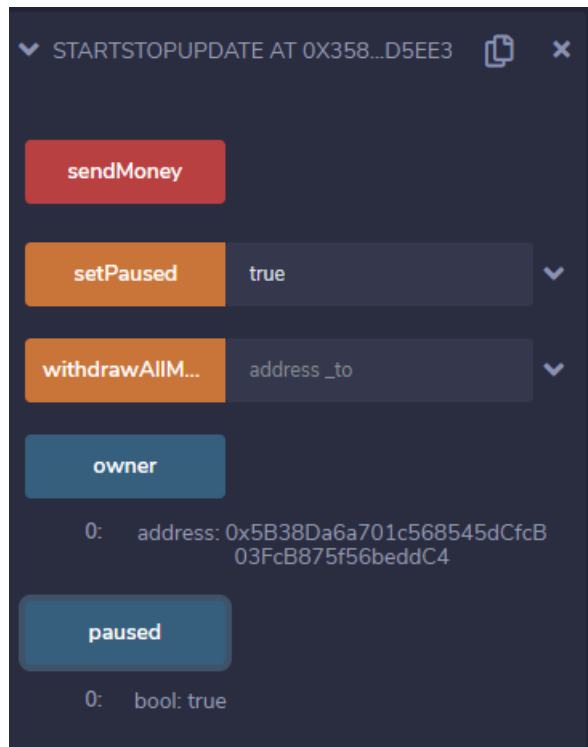
+ Hàm setPaused(): Hàm này có chức năng để cập nhật biến bị tạm dừng. Chức năng này chỉ có thể được gọi bởi chủ sở hữu.

Và rút tiền chỉ có thể được gọi nếu hợp đồng không bị tạm dừng.

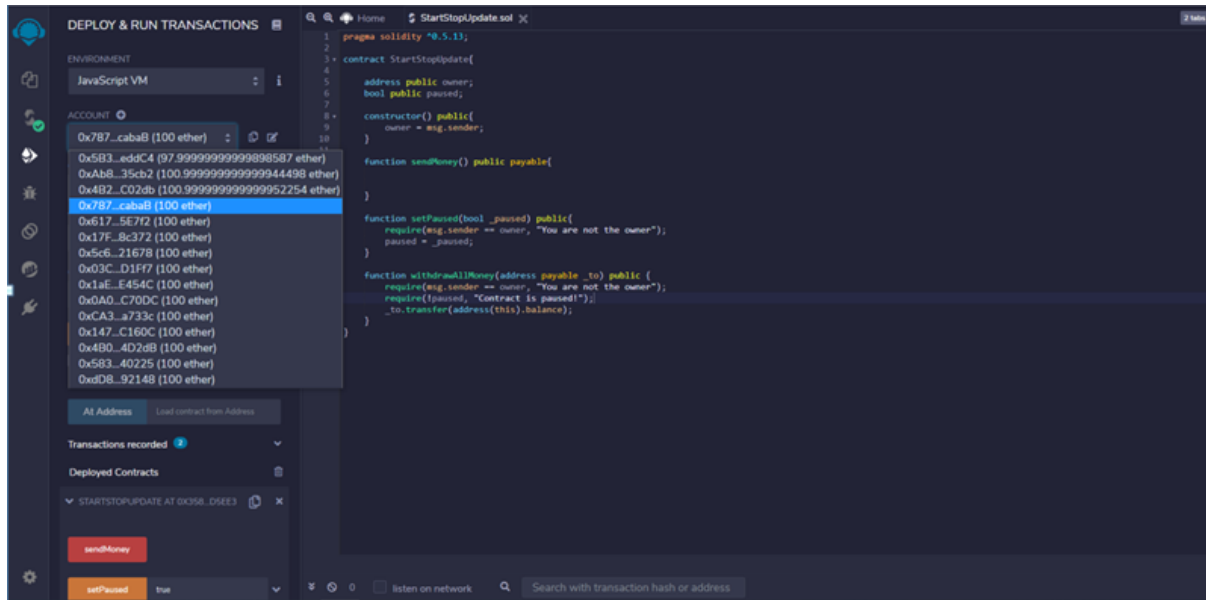
Ta tiến hành “Deploy” Contract sẽ hiển thị.



Bây giờ , chúng ta thử tiến hành cập nhập lại paused thành true

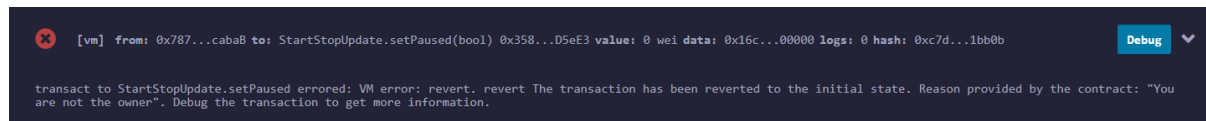


Giờ thì hãy thử chọn bất kỳ tài khoản nào để xem khi setPaused lại nó hoạt động như nào nha



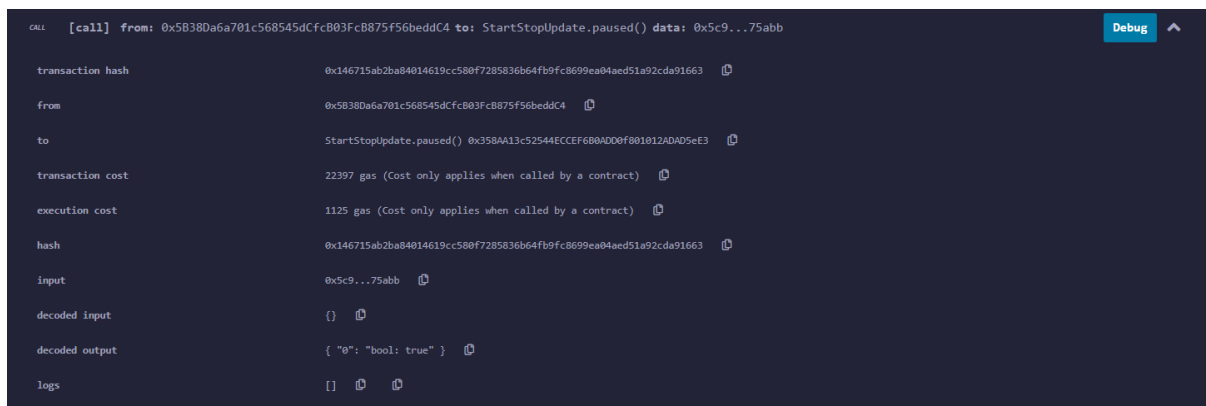
Mình chọn tài khoản thứ 4 nhé.

Khi thay đổi trường setPaused thành false và nhấn setPaused và kết quả là:



Thông báo lỗi sẽ hiển thị.

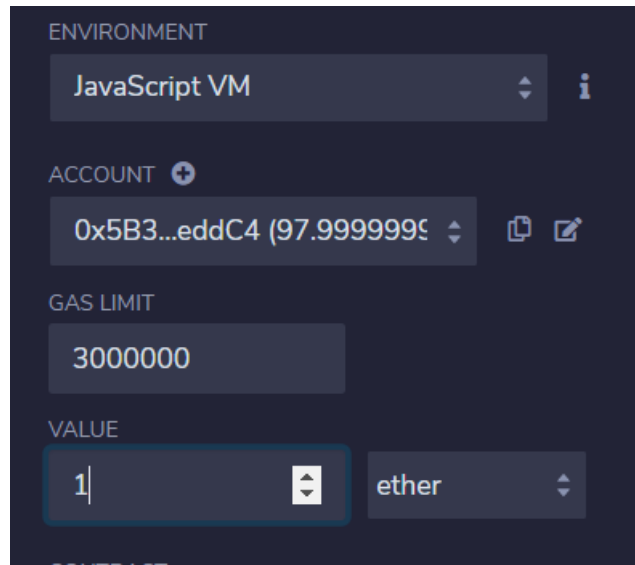
Để fix lỗi này ta chỉ cần chuyển trở về địa chỉ ban đầu và khi đó ra sẽ setPaused được.



Giờ cùng thử gửi và rút tiền nhé!

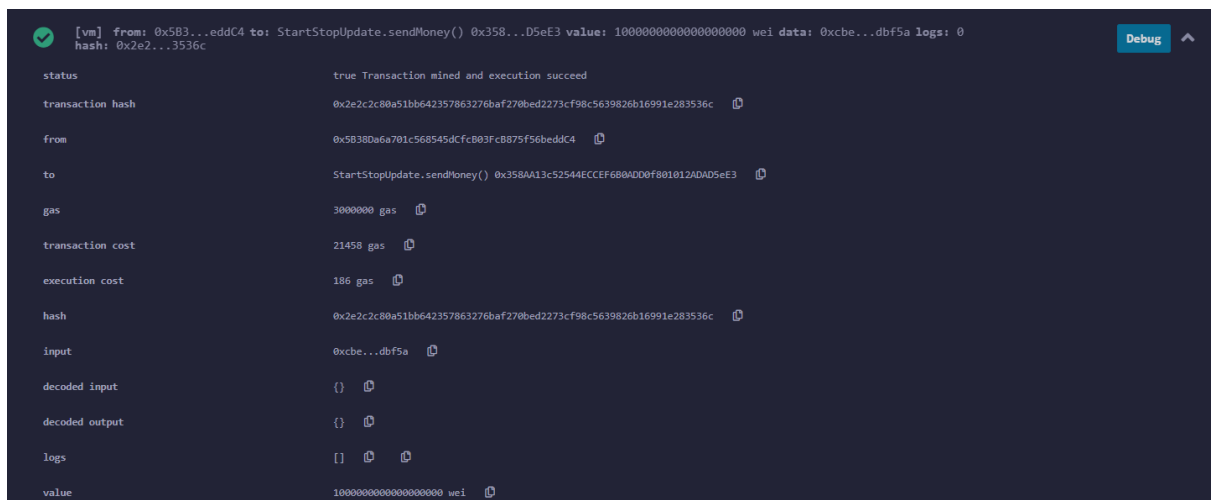
Chúng ta cũng thực hiện tương tự như những lần trước.

Gửi tiền: Nhập giá trị vào trường Value.

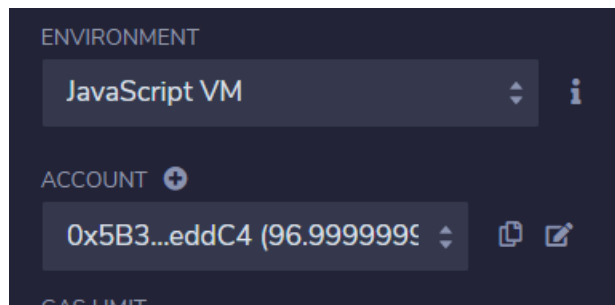


The screenshot shows the 'ENVIRONMENT' configuration panel of a JavaScript VM. It includes fields for 'ACCOUNT' (0x5B3...eddC4 with a balance of 97.9999999 ether), 'GAS LIMIT' (3000000), and 'VALUE' (1 ether). The 'VALUE' field is highlighted with a red box.

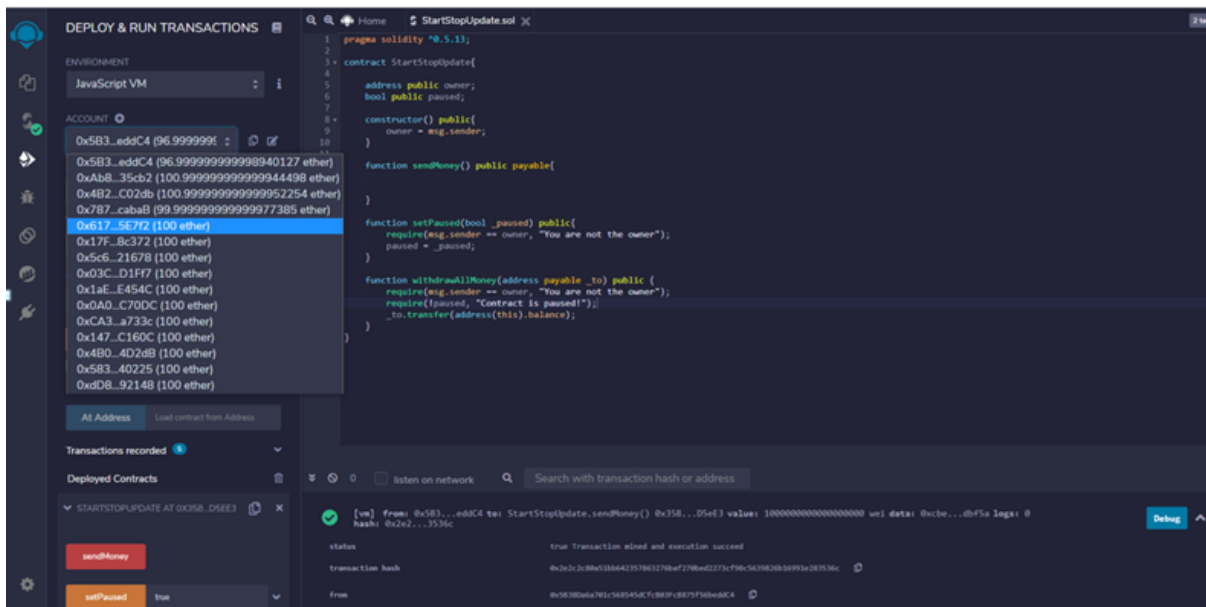
Và nhấn sendMoney thông báo giao dịch thành công sẽ hiển thị.



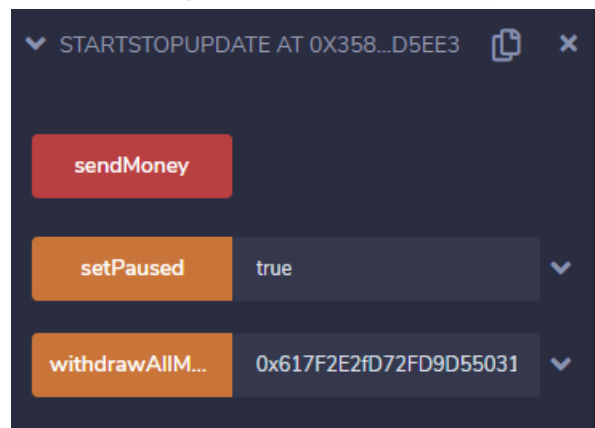
Và số ether trong tài khoản sẽ bị trừ 1 ether. Gửi tiền thành công.



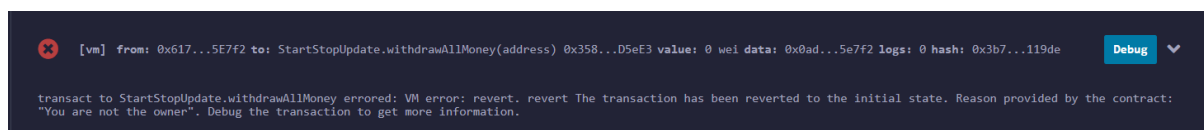
Rút tiền: Ta chọn 1 địa chỉ trong danh sách tài khoản.



Copy địa chỉ và dán vào trường withdrawAllMoney

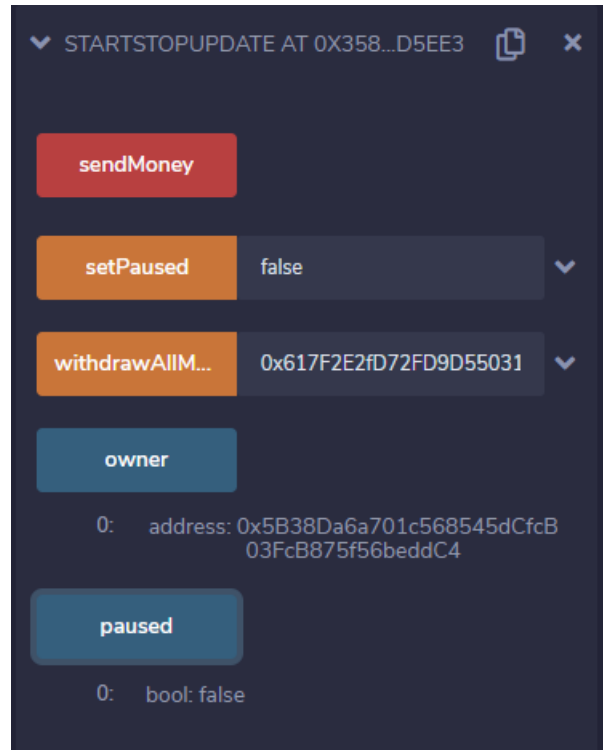


Nhấn vào withdrawAllMoney để xem điều gì xảy ra.

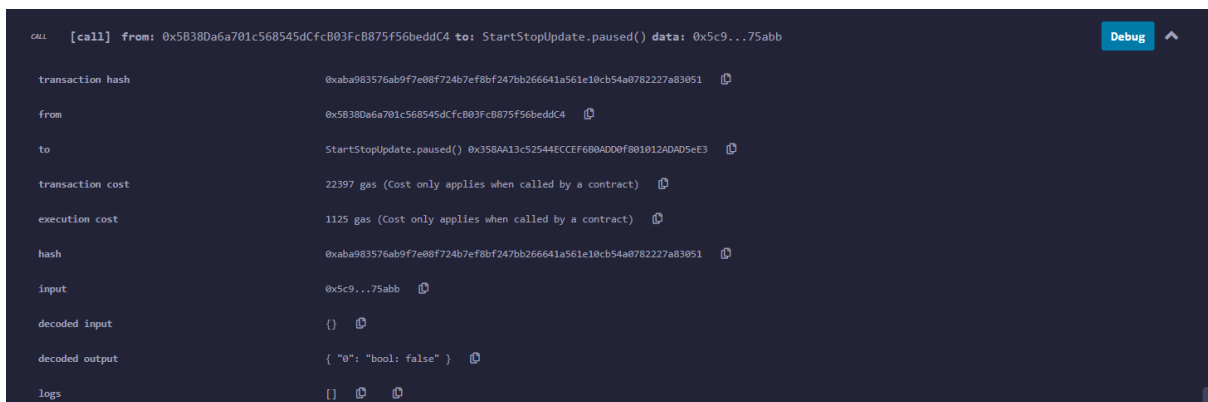


Thông báo hiển thị lỗi.

Giờ để gỡ lỗi này, ta chuyển địa chỉ lại địa chỉ ban đầu và thực hiện setPaused và rút tiền để xem kết quả.



Bây giờ nếu muốn biết mọi thứ đã thành công hay chưa ta cùng xem thông báo hiển thị như nào trong bảng biểu của Remix nhé. Đây là kết quả nè!



✓

[vm] from: 0x583...eddC4 to: StartStopUpdate.withdrawAllMoney(address) 0x358...D5eE3 value: 0 wei data: 0x0ad...5e7f2 logs: 0 hash: 0x143...71489

Debug ^

status	true Transaction mined and execution succeed
transaction hash	0x1437cc56f6314b02e4b2252bdea9dc9284f19720d62fb91023a9e04f67671489 🔗
from	0x5838da6a701c568545dcfc803fc8875f56beddc4 🔗
to	StartStopUpdate.withdrawAllMoney(address) 0x358AA13c52544ECCEf600A0D0fB01012ADAD5eE3 🔗
gas	3000000 gas 🔗
transaction cost	32877 gas 🔗
execution cost	10197 gas 🔗
hash	0x1437cc56f6314b02e4b2252bdea9dc9284f19720d62fb91023a9e04f67671489 🔗

Tiếp theo ta đến với phương thức hủy bỏ 1 Smart Contract (Destroy).

Chúng ta không thể xóa thông tin khỏi Blockchain, nhưng chúng ta có thể cập nhật trạng thái hiện tại để ta không thể tương tác với một địa chỉ nữa trong tương lai. Khi quá trình tự cấu trúc được gọi, bạn không thể tương tác với Hợp đồng thông minh nữa.

```
function destroySmartContract(address payable _to) public {  
    require(msg.sender == owner, "You are not the owner");  
    selfdestruct(_to);  
}
```

+ Khi “selfdestruct” được gọi, tất cả số tiền còn lại trên địa chỉ của Hợp đồng thông minh sẽ được chuyển đến địa chỉ đó.

Sau khi bạn gọi destroySmartContract, địa chỉ của Hợp đồng thông minh sẽ không chứa mã nữa. Bạn vẫn có thể gửi các giao dịch đến địa chỉ và chuyển Ether đến đó, nhưng sẽ không có bất kỳ mã nào có thể gửi lại Ether cho bạn.

Bây giờ, ta tiến hành triển khai Smart Contract.

Gửi đi một số Ether . Minh nhập 5ether vào trường Value.

ENVIRONMENT
JavaScript VM

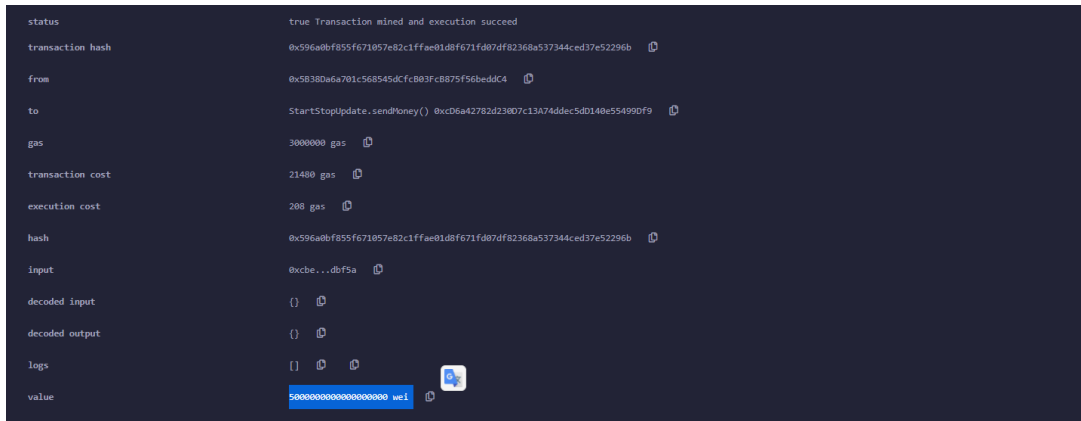
ACCOUNT +
0x5B3...edc4 (96.999999%)

GAS LIMIT
3000000

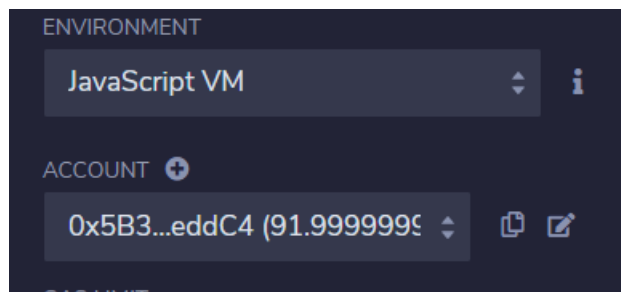
VALUE
5 ether

CONTRACT

Sau đó, kéo xuống và nhấp vào sendMoney để gửi ether.

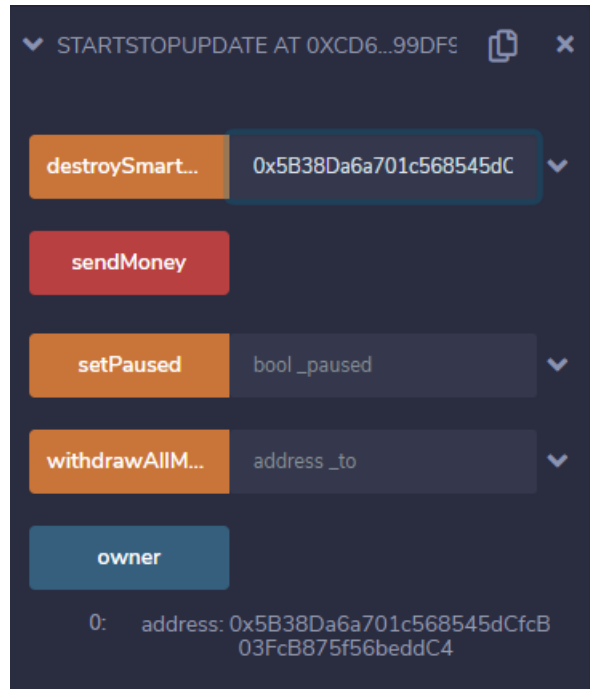


Lúc này, thông báo giao dịch thành công. Tài khoản sẽ bị trừ 5ether.

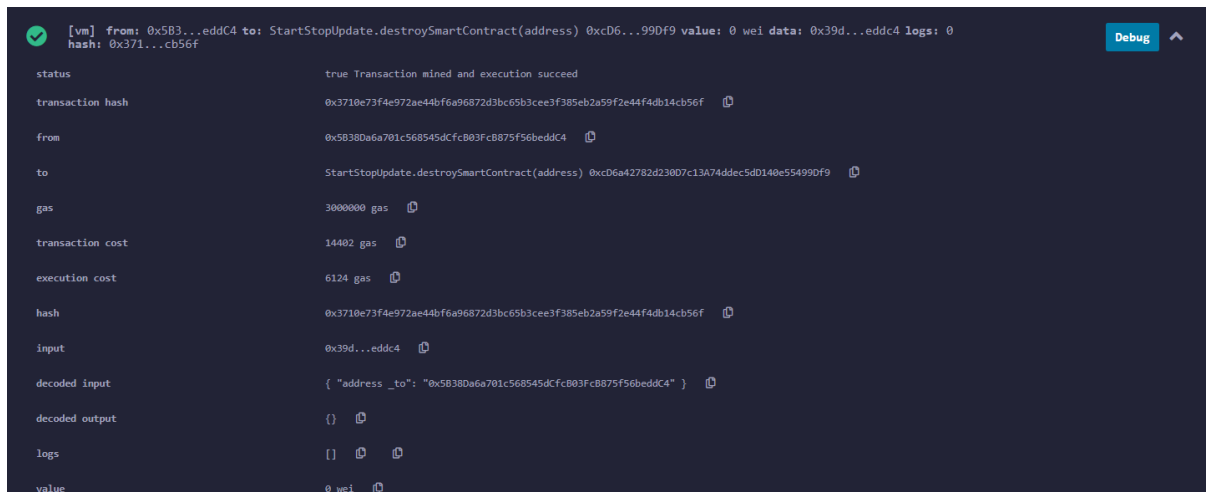


Bây giờ thử gọi destroySmartContract và cung cấp Tài khoản của riêng bạn, để Ether được gửi lại.

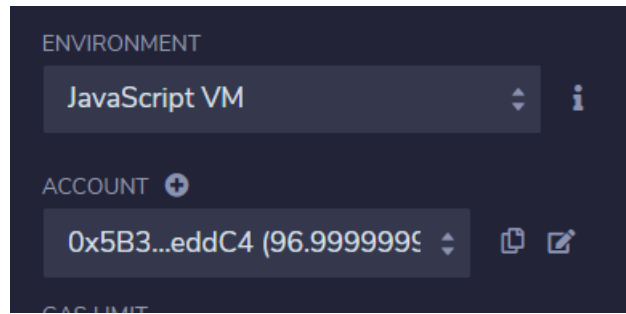
Ta copy địa chỉ owner của mình và bỏ vào trường destroySmartContract và tiến hành nhấn nút destroySmartContract.



Khi đó thông báo hiển thị thành công.



Và kết quả là 5ether đã được trả lại cho tài khoản đó.



LAB 8:

Đầu tiên, ta tạo 1 folder mới và trong folder đó tạo 1 file với tên tùy ý ở đây mình đặt là SimpleMapping.sol

Trước hết, ta cần khai báo phiên bản trong Solidity là phiên bản bao nhiêu.

Ta dùng pragma:

```
pragma solidity ^0.6.0;
```

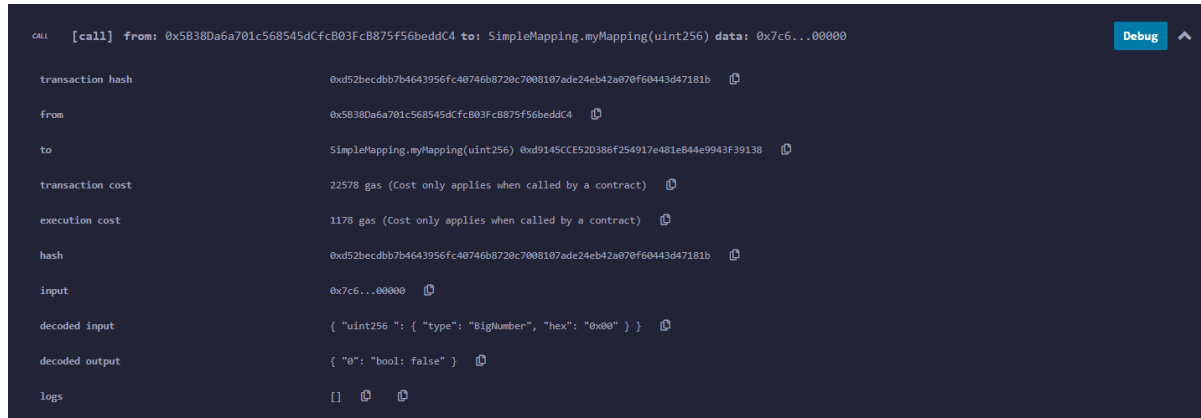
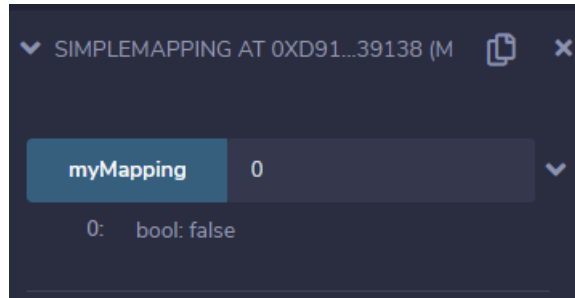
Tiếp theo, ta tạo một contract với tên SimpleMapping{}

```
contract SimpleMapping{
    mapping(uint => bool) public myMapping;
}
```

Ta khai báo một biến kiểu mapping: với loại khóa là uint và loại giá trị là bool

Tiến hành “Deploy” thông báo sẽ hiển thị giao dịch thành công.

Kéo xuống mục Deployed Contracts, để xem kết quả như nào ta không để trống trường myMapping hãy thử nhập 1 giá trị bất kì vào ô myMapping và nhấp gọi myMapping gọi giá trị sẽ trả về giá trị mặc định false. Vì chưa có bất kì hàm nào để có thể thay đổi được giá trị bool.



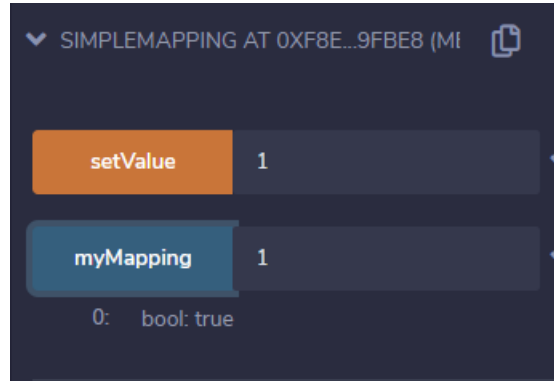
Bây giờ, ta thực hiện tạo 1 hàm setValue

```
function setValue(uint _index) public{
    myMapping[_index] = true;
}
```

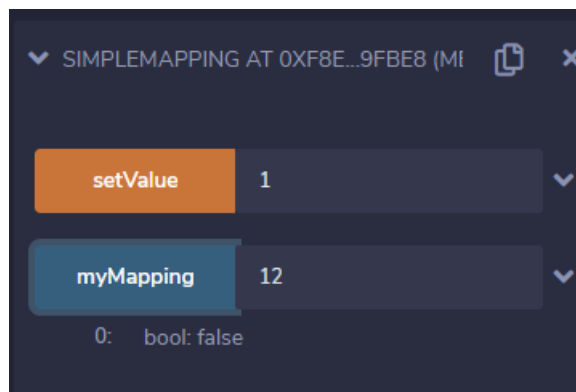
Hàm này cho thấy là khi ta nhập giá trị của setValue khớp với giá trị ánh xạ myMapping thì bool sẽ trả về true.

Giờ ta tiến hành “Deploy” và thông báo sẽ hiển thị giao dịch thành công.

Kéo xuống và nhập giá trị vào 2 ô setValue và myMapping. Nếu nhập khớp nhau thì kết quả trả về true. Như ví dụ dưới đây:



Giờ hãy thử nhập 2 giá trị khác nhau xem sao:



Như vậy đã rõ ràng. Khi nhập khác giá trị bool sẽ trả về false.

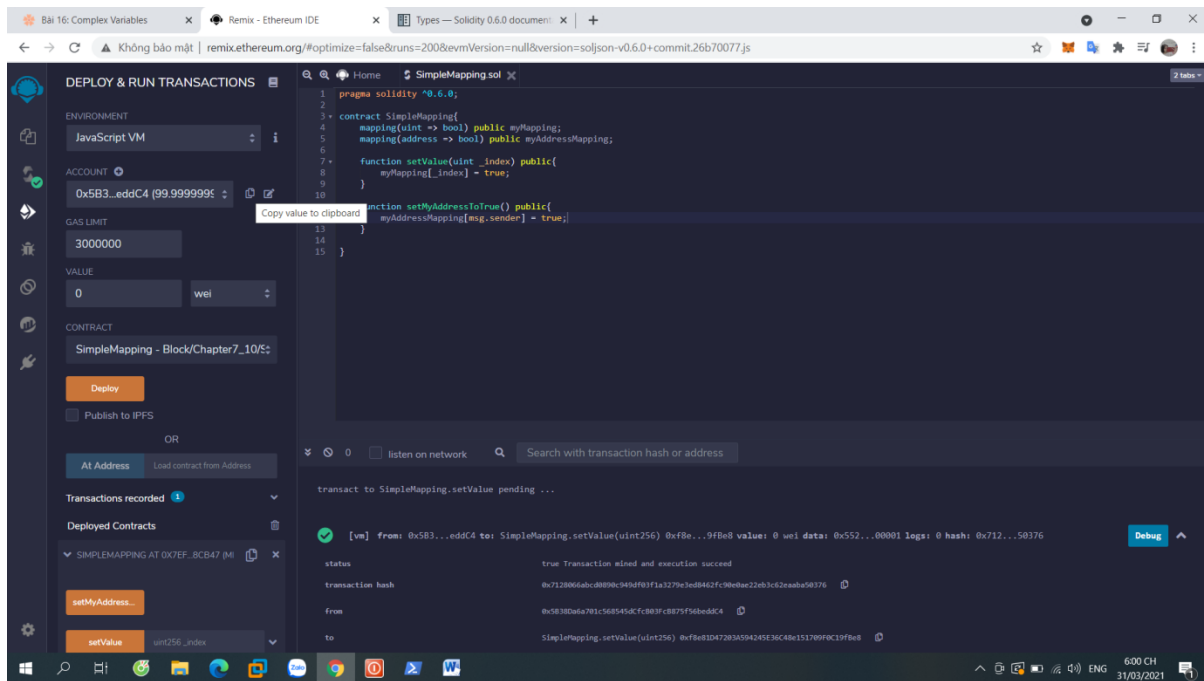
Tiếp theo, ta cũng khai báo biến kiểu mapping nhưng lần này ta đổi loại khóa từ uint sang address và loại giá trị vẫn là bool.

```
contract SimpleMapping{
    mapping(uint => bool) public myMapping;
    mapping(address => bool) public myAddressMapping;
    function setValue(uint index) public{
```

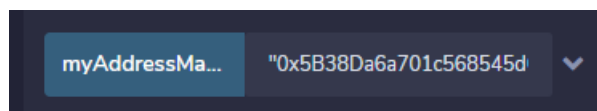
Tạo 1 function setMyAddressToTrue

```
function setMyAddressToTrue() public{
    myAddressMapping[msg.sender] = true;
}
```

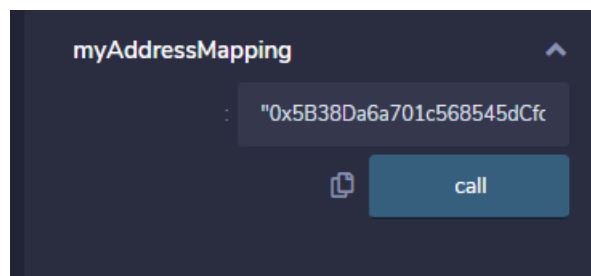
Hàm này có chức năng khi ta chèn vào một địa chỉ trong myAddressMapping và bắt đầu giao dịch setMyAddressToTrue thì giá trị sẽ trả về true. Cùng thực hành nhé!.



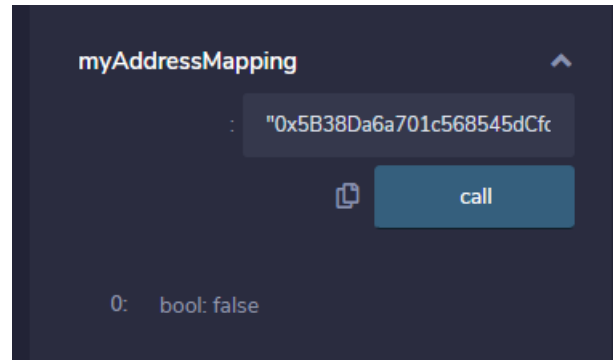
Copy địa chỉ và kéo xuống dán vào trường myAddressMapping



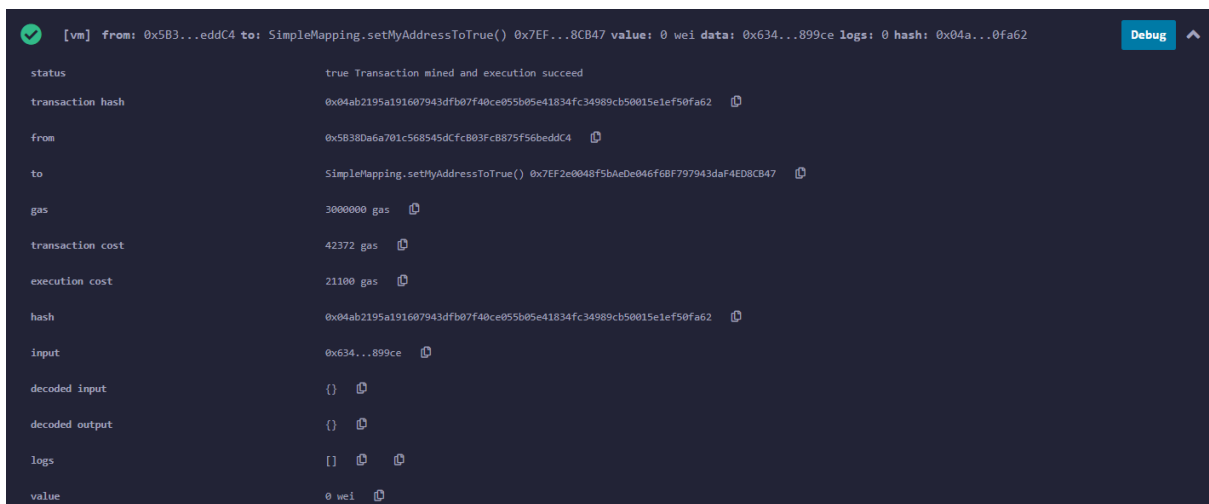
Giờ ta tiến hành gọi myAddressMapping



Và kết quả trả về là false.

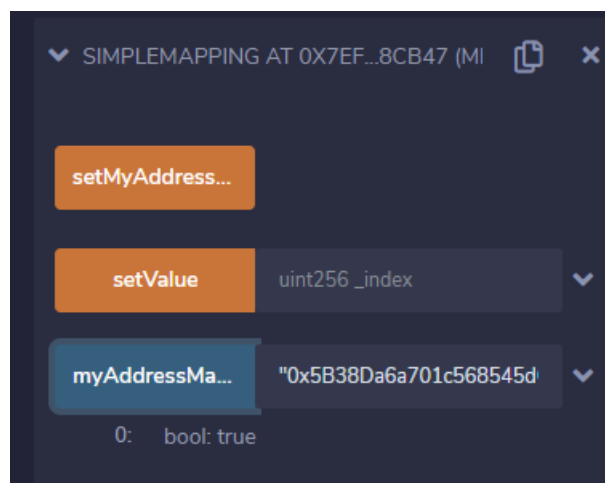


Bây giờ, ta thực thi hàm setMyAddressToTrue



Khi đó, bảng biểu Remix đã hiển thị giao dịch thành công.

Lúc này, ta nhấp vào lại myAddressMapping



Như vậy ta đã hàm setMyAddress đã thực thi thành công và cho ta giá trị là true.

LAB 9:

Đầu tiên tạo 1 folder và tạo 1 file mới trong folder đó.

Trước khi thực hiện cần khai báo phiên bản đang sử dụng trong Solidity có version bao nhiêu. Ta dùng pragma

```
pragma solidity ^0.5.13;  
  
contract MappingStruct{
```

Tiếp theo, ta tạo 1 Smart Contract có tên MappingStruct{}

Ta tiến hành tạo các function cho Contract

```
function getBalance() public view returns(uint){  
    return address(this).balance;  
}  
  
function sendMoney() public payable{  
  
}  
  
function withdrawAllMoney(address payable _to) public {  
    _to.transfer(address(this).balance);  
}
```

Ta có thể thấy các function này tương tự như các function ở Lab 6.

Với hàm đầu tiên là hàm có chức năng lấy ra số dư của địa chỉ đó bằng câu lệnh address(this).balance và trả về số dư.

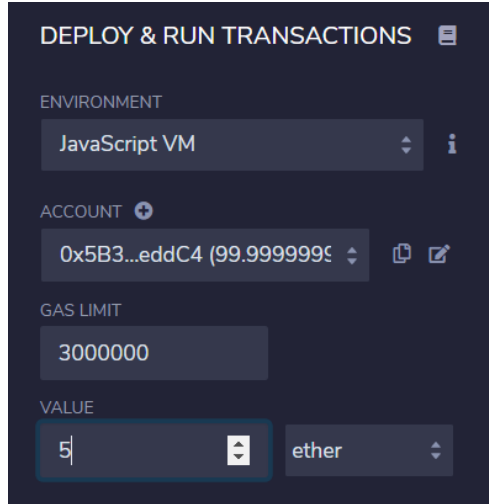
Ở function thứ 2 có chức năng gửi đi số ether.

Function thứ 3 có chức năng gửi tiền đến một địa chỉ cụ thể. Ta có thể chỉ định một địa chỉ và chuyển tiền đến địa chỉ đó.

Bây giờ, ta tiến hành “Deploy” contract. Thông báo giao dịch thành công sẽ hiển thị ở bảng biểu của Remix.

Giờ ta thực thi các hàm xem sao.

Thử gửi 5ether đến Hợp đồng thông tin



DEPLOY & RUN TRANSACTIONS

ENVIRONMENT

JavaScript VM

ACCOUNT

0x5B3...eddC4 (99.999999 ether)

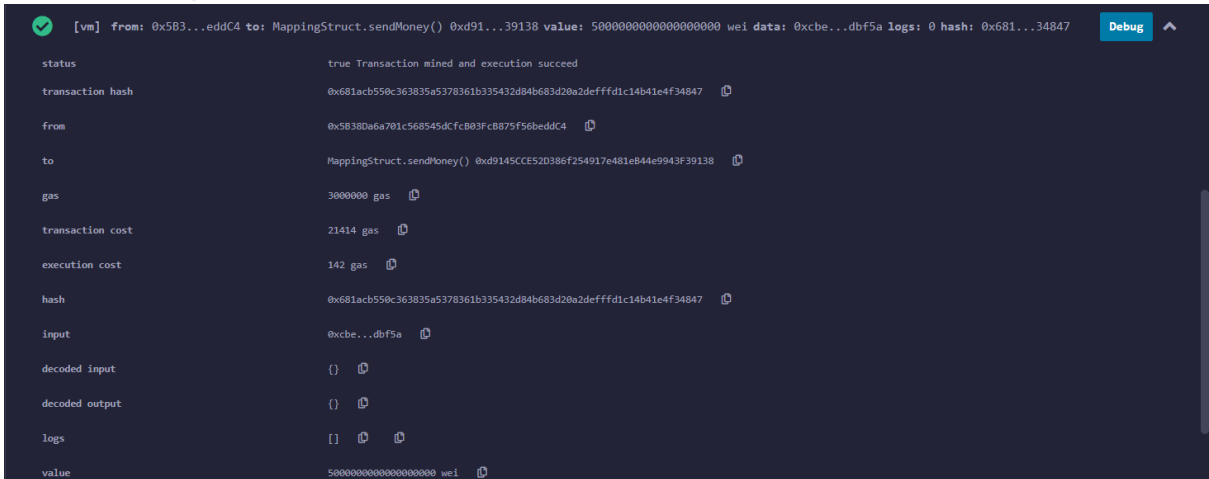
GAS LIMIT

3000000

VALUE

5 ether

- Kéo xuống và nhấp vào sendMoney. Sau khi nhấp vào sendMoney thông báo giao dịch thành công sẽ hiển thị.



[vm] from: 0x5B3...eddC4 to: MappingStruct.sendMoney() @xd91...39138 value: 5000000000000000000 wei data: 0xcbe...dbf5a logs: 0 hash: 0x681...34847 Debug

status: true Transaction mined and execution succeed

transaction hash: 0x681acb550c363835a5378361b335432d84b683d20a2defffd1c14b41e4f34847

from: 0x58380a6a701c568545dcfc803fc8875f56beddC4

to: MappingStruct.sendMoney() @xd9145CCE52D386f254917e481eB44e9943f39138

gas: 3000000 gas

transaction cost: 21414 gas

execution cost: 142 gas

hash: 0x681acb550c363835a5378361b335432d84b683d20a2defffd1c14b41e4f34847

input: 0xcbe...dbf5a

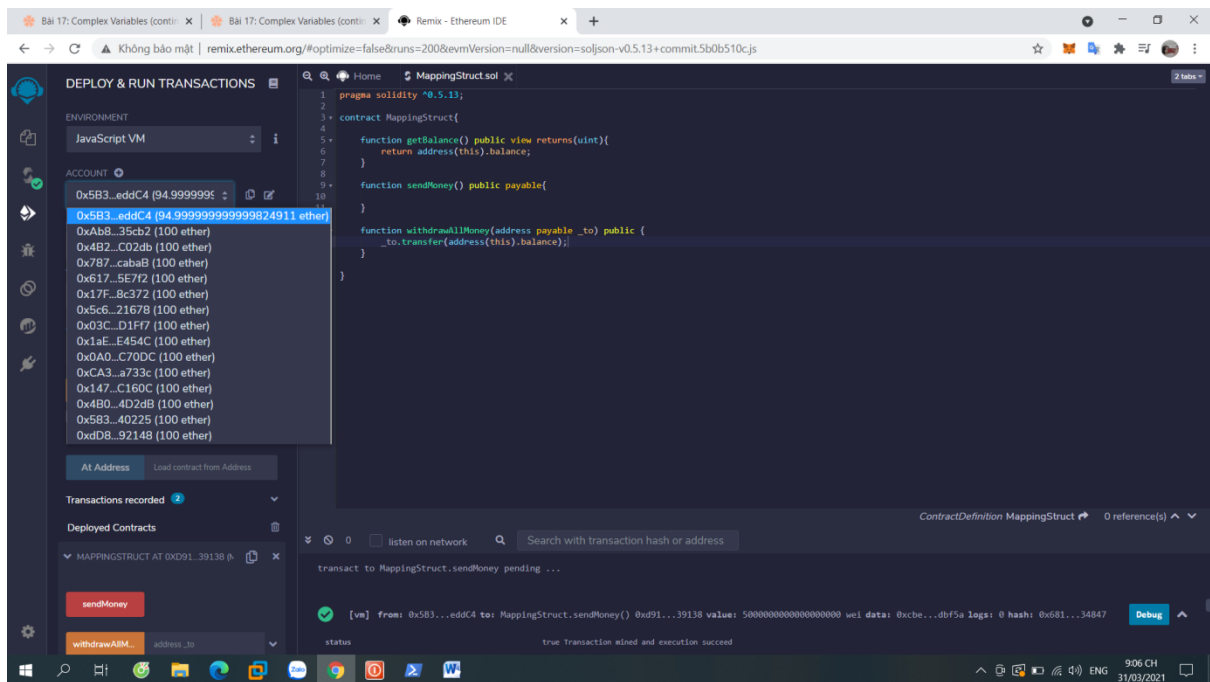
decoded input: {}

decoded output: {}

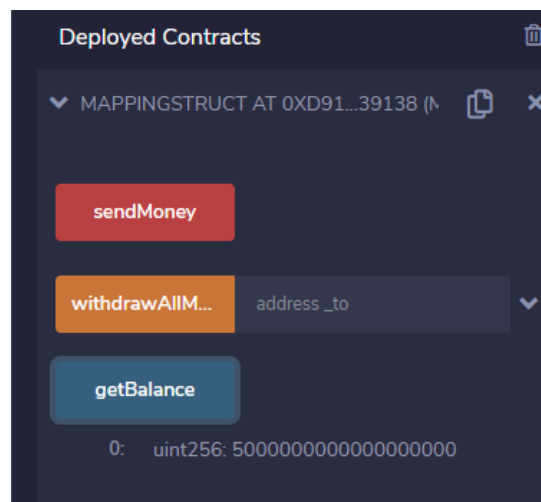
logs: []

value: 5000000000000000000 wei

Và kết quả sẽ thấy rõ khi ta nhìn vào số ether trên danh sách địa chỉ.



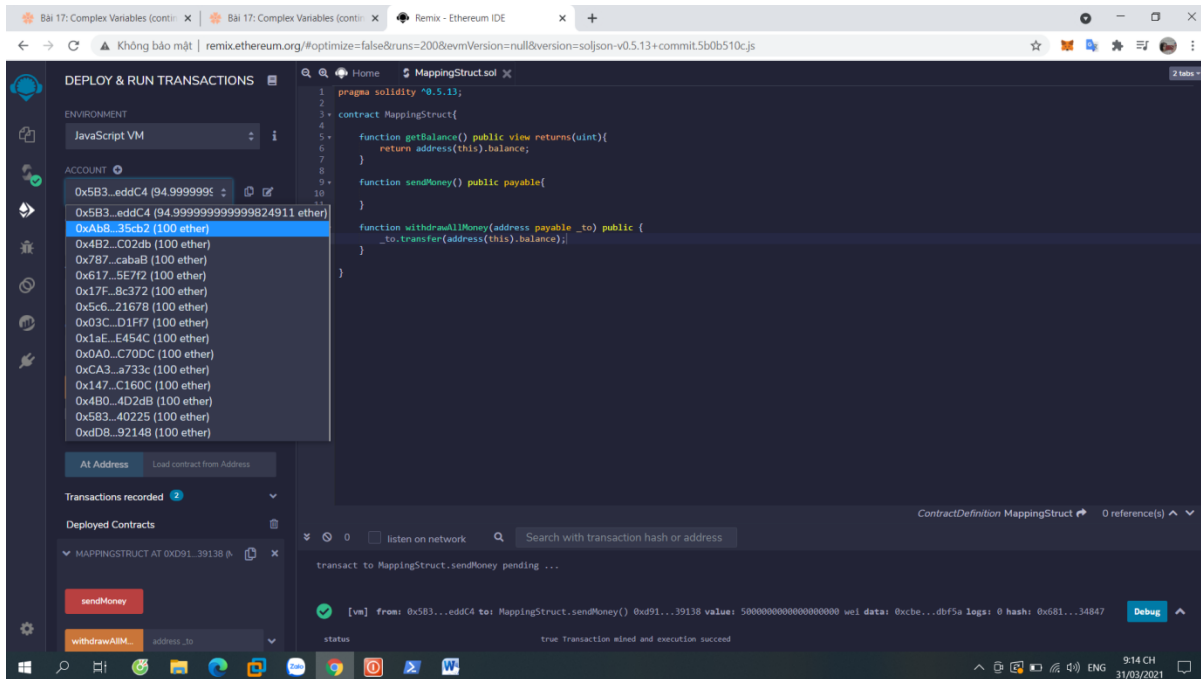
Và cùng xem số dư trả về có đúng với số ether mình gửi không nhé. Ta kéo xuống và nhấp vào getBalance để xem.



Như vậy, kết quả trả về hoàn toàn chính xác. Sở dĩ, ta thấy quá nhiều 0 đến thế là do $1 \text{ ether} = 10^{18} \text{ wei}$ đó nhé.

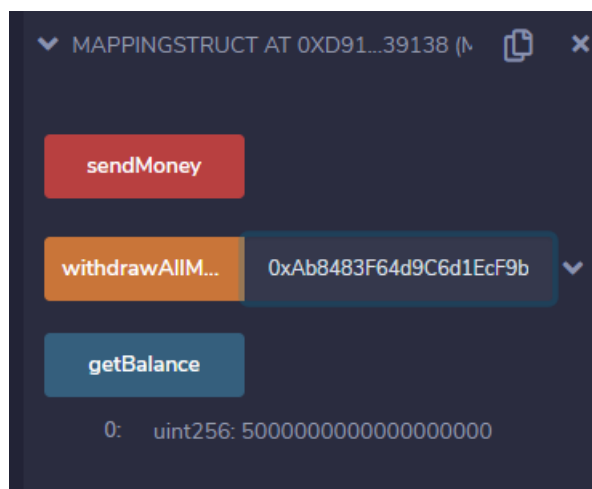
Bây giờ, thực hiện chuyển tiền.

Ta chọn một địa chỉ bất kì trong danh sách Account. Mình chọn địa chỉ thứ 2 từ trên xuống.

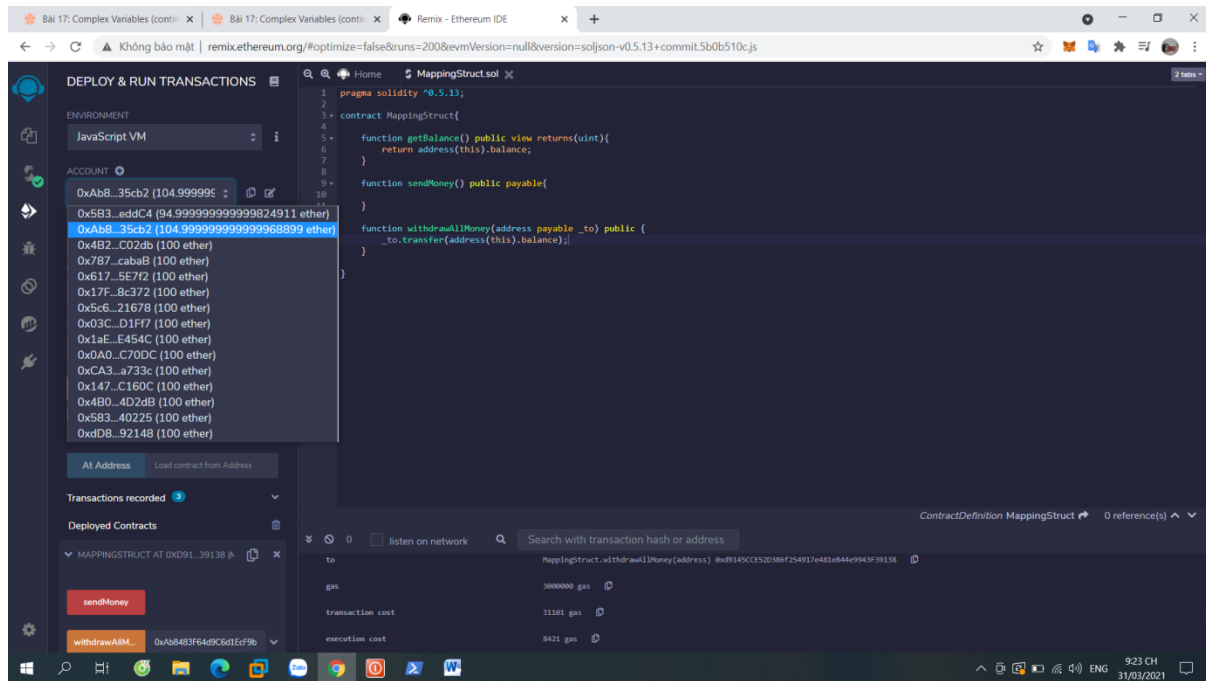


Và nhân sao chép địa chỉ đó. Chắc hẳn các bạn phải biết sao chép như nào rồi chứ!

Giờ thì dán địa chỉ đó vào trường withdrawAllMoney.

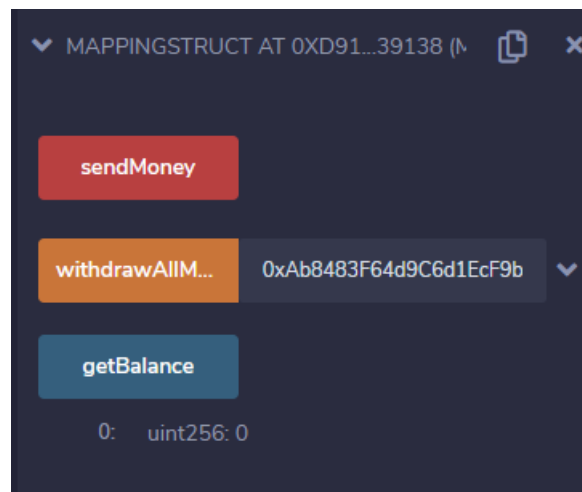


Sau đó nhấp vào nút `withdrawAllMoney` để giao dịch được gửi đi. Giao dịch thành công sẽ hiển thị thông báo trong bảng biểu.



Khi đó, địa chỉ thứ 2 đã được chuyển vào 5ether nhưng do hao phí về gas nên ở đây ta chỉ nhìn thấy 4.99999999..ether.

Lúc này, hãy kiểm tra lại số dư hiện tại, ta nhấp vào `getBalance`:



Và số dư bây giờ sẽ trở về 0.

Tiếp theo, ta update code lên tí nhé.

```
mapping(address => uint) public balanceReceived;

function getBalance() public view returns(uint){
    return address(this).balance;
}

function sendMoney() public payable{
    balanceReceived[msg.sender] += msg.value;
}

function withdrawAllMoney(address payable _to) public {
    uint balanceToSend = balanceReceived[msg.sender];
    balanceReceived[msg.sender] = 0;
    _to.transfer(balanceToSend);
}
```

Thêm 1 biến kiểu mapping với loại khóa là address và loại giá trị là uint, ánh xạ địa chỉ Ethereum thành giá trị số nguyên không dấu.

Trong phương thức sendMoney:

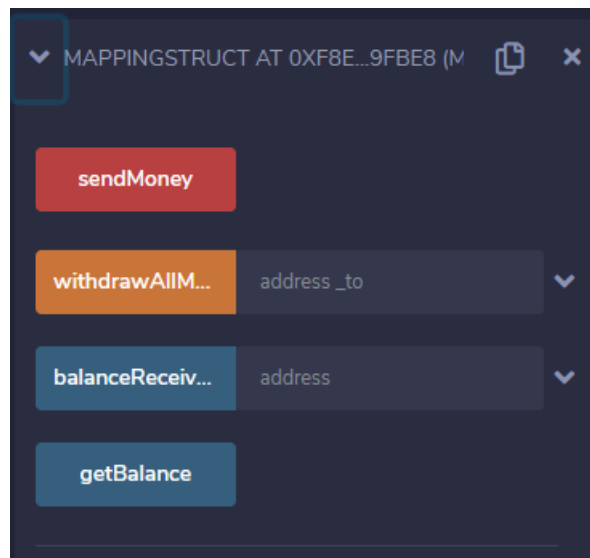
Ta thực hiện việc địa chỉ được chỉ định nhận được số dư khớp với giá trị được gửi. Cứ mỗi lần thực hiện gửi ether số dư trong tài khoản sẽ được cộng dồn vào.

Với function withdrawAllMoney:

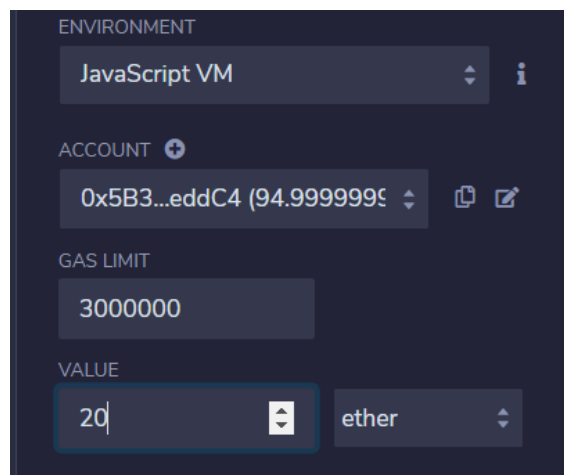
Ở đây ta thêm biến uint với balanceToSend sẽ nhận được số dư từ địa chỉ đó.

Giờ ta tiến hành triển khai hợp đồng, ta nhấn vào “Deploy”. Giao dịch thành công.

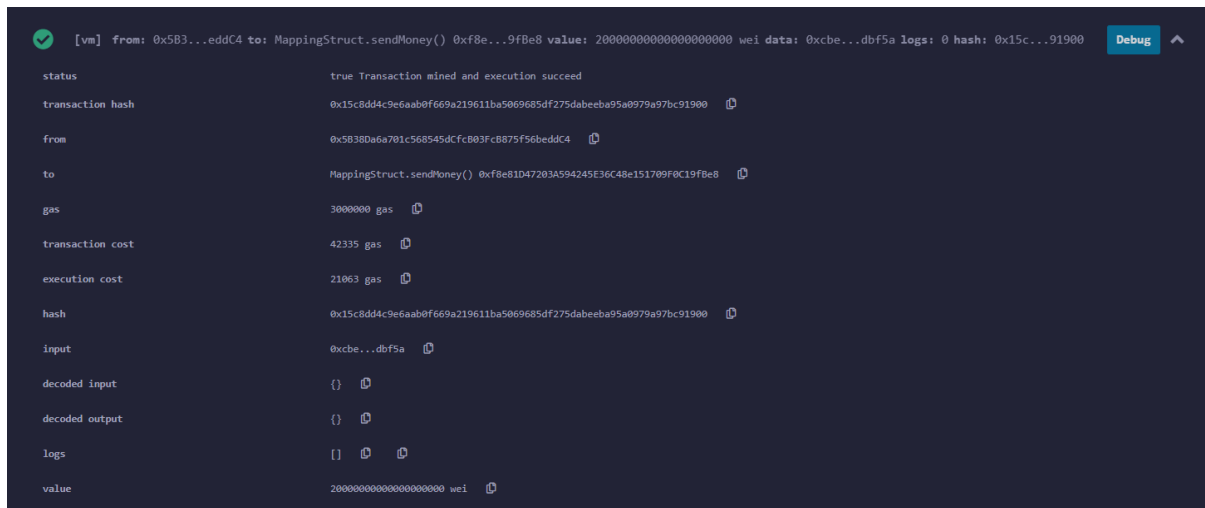
Mục Deployed Contract sẽ hiển thị như bên dưới:



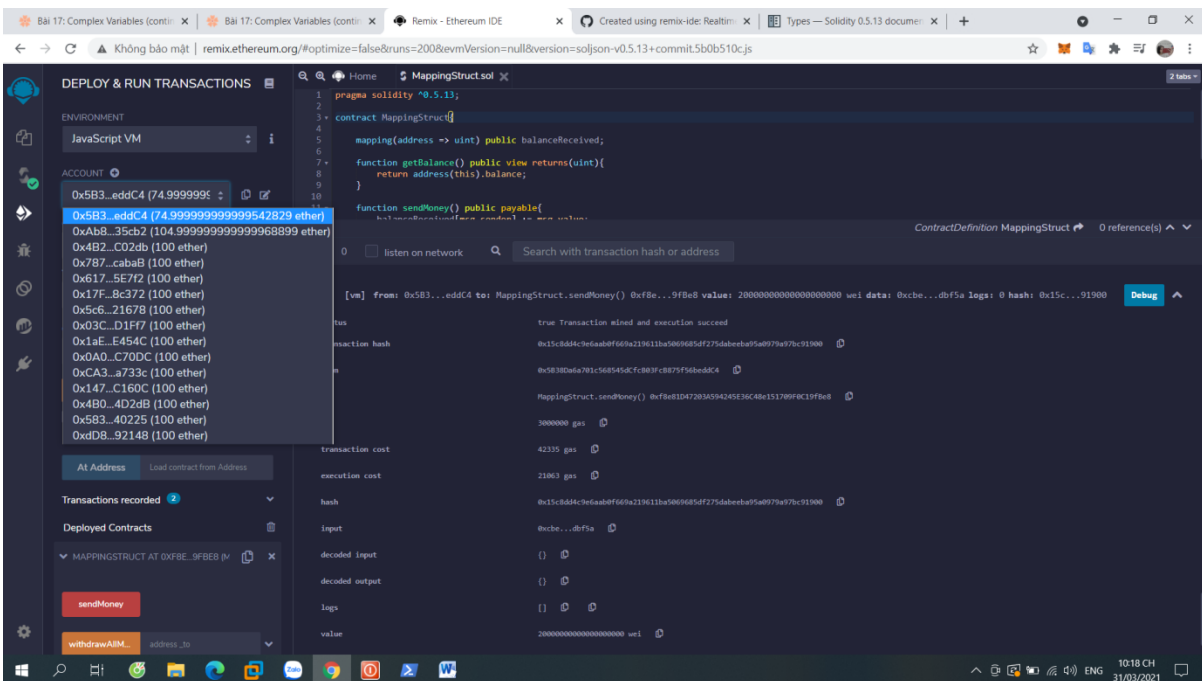
Lúc này, ta thử gửi 1 số lượng ether đến Hợp đồng. Ở đây, mình nhập vào 20ether.



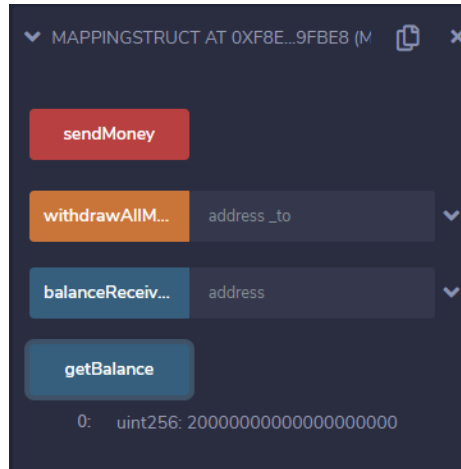
Kéo xuống và nhấn vào sendMoney. Khi đó, thông báo sẽ hiển thị giao dịch thành công.



Và kết quả số lượng ether đã bị trừ bớt.

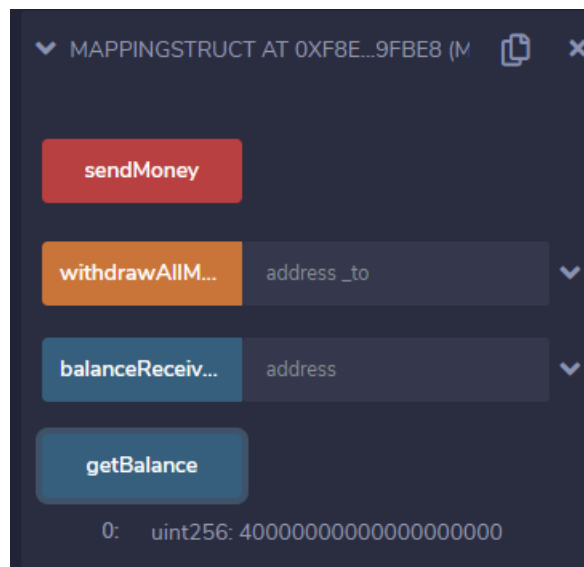


Và bây giờ số dư hiện tại đang có sẽ là:

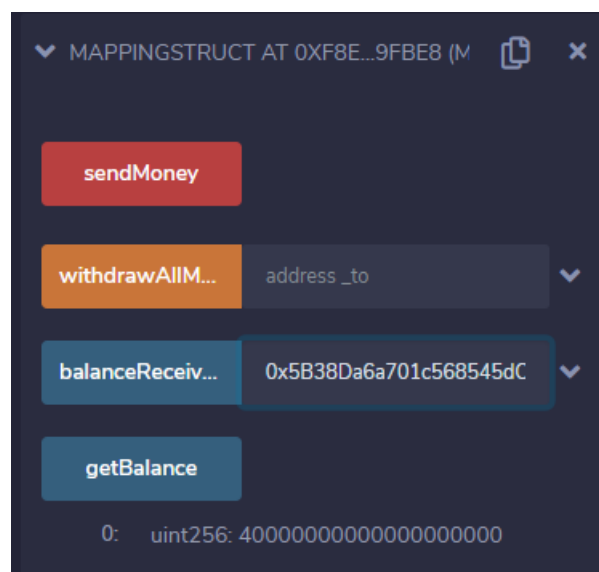


Hãy thử thêm một lần nữa xem số ether có tăng dồn lên không nhé!.

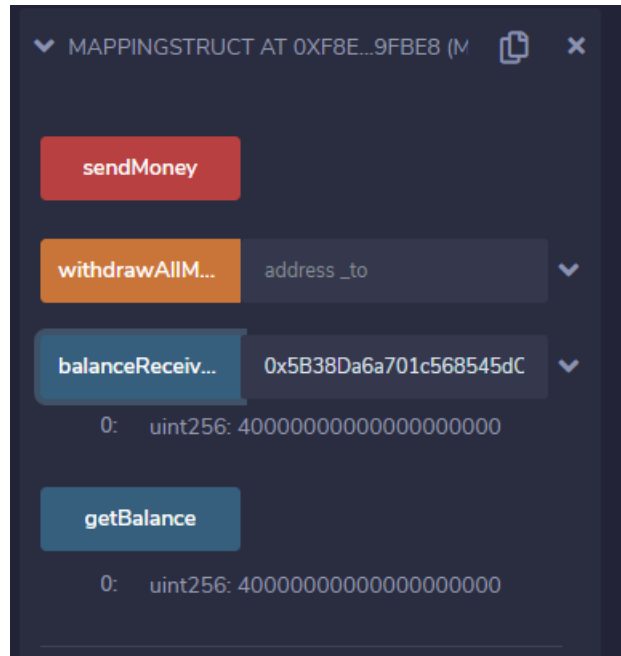
Ta thực hiện tương tự, nhập giá trị vào ô Value và nhấp vào sendMoney. Khi giao dịch thành công số ether trong địa chỉ đó sẽ giảm đi và số dư getBalance sẽ trả về. Ở đây, mình đã gửi đi 20ether nữa và như vậy số dư lúc này đã cộng dồn lên thành 40ether.



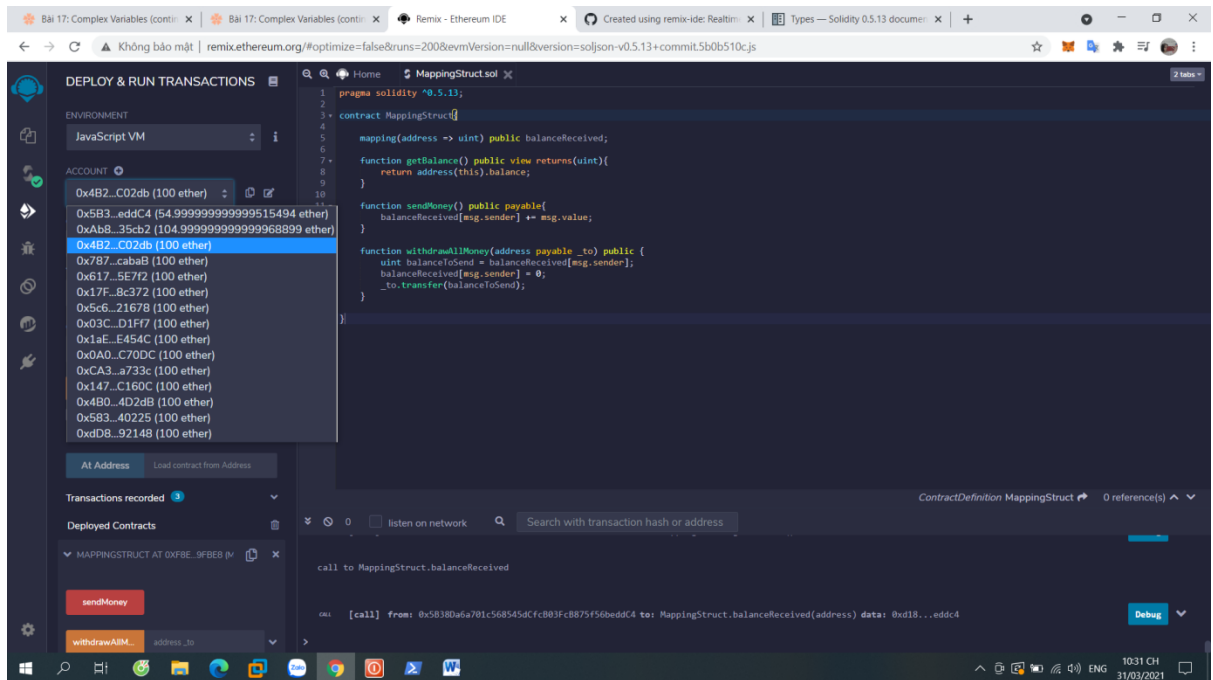
Để xem số dư nhận được từ địa chỉ đó là bao nhiêu thì ta thực hiện bằng cách sao chép địa chỉ đó và dán vào trường balanceReceived.



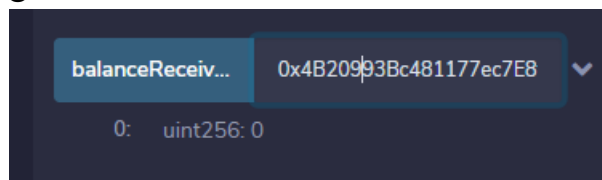
Sau khi dán vào, ta nhấn nút `balanceReceived` và kết quả từ địa chỉ đó trả về:



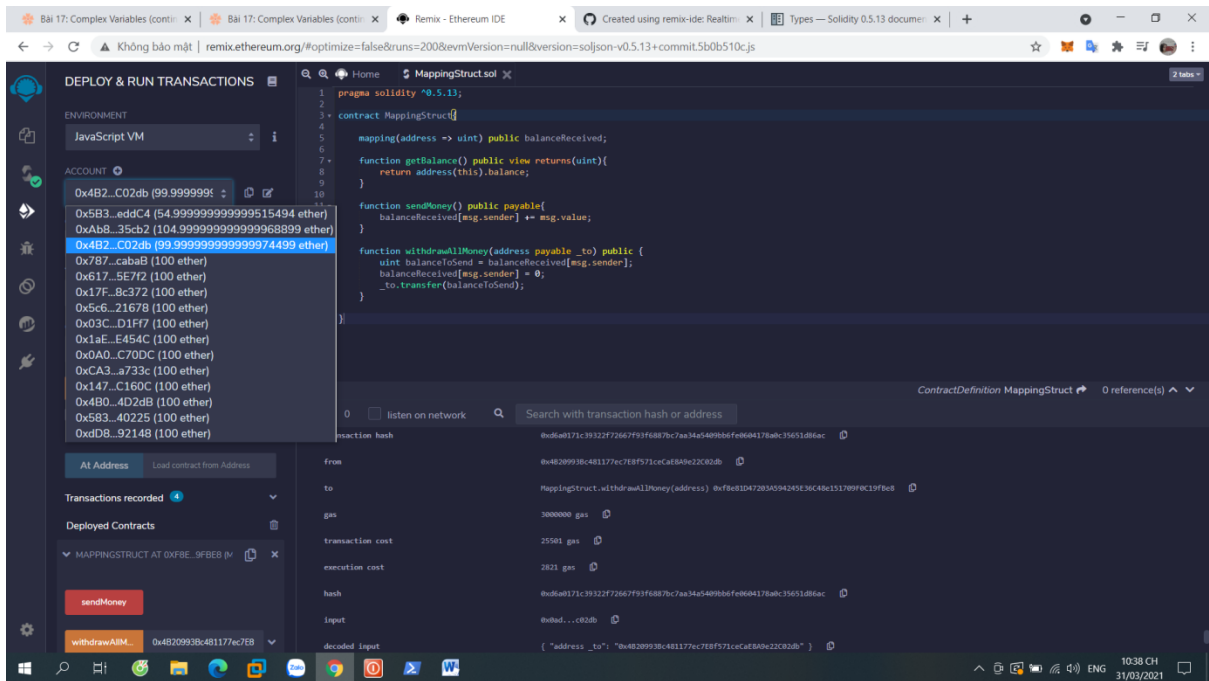
Nếu muốn xem số dư đã nhận được từ địa chỉ khác ta chỉ cần sao chép địa chỉ và dán vào ô balanceReceived. Ở đây, ta thử chọn địa chỉ thứ 3 và dán vào để xem kết quả:



Khi gọi balanceReceived kết quả sẽ trả về 0 vì ta chưa thực hiện 1 giao dịch nào từ địa chỉ nên không có số dư.

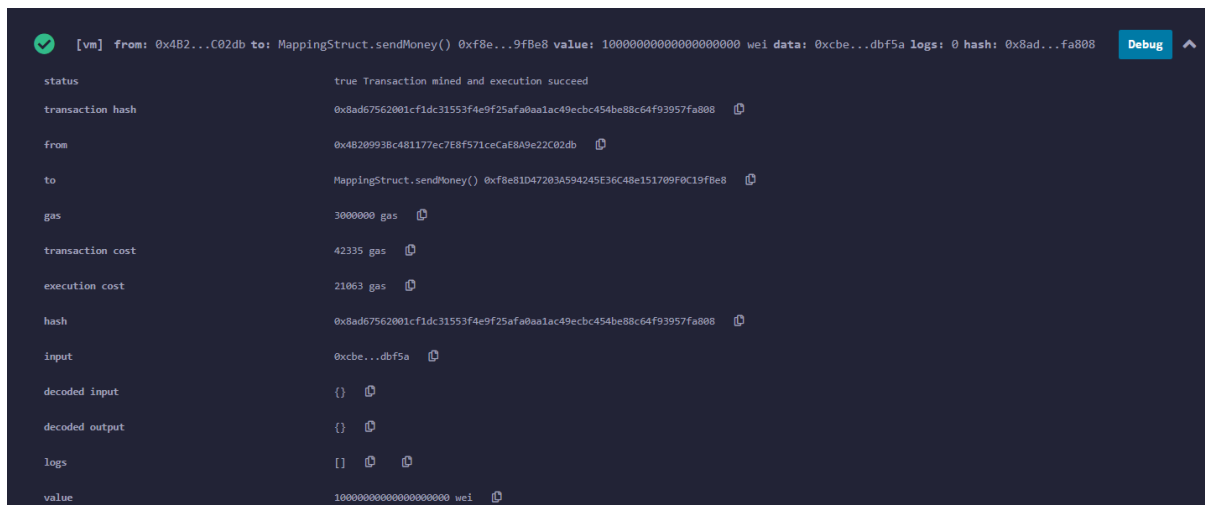


Cũng từ địa chỉ thứ 3 được sao chép đó ta dán vào trường withdrawAllMoney. Và nhấn vào nút withdrawAllMoney giao dịch sẽ thực hiện.



Ta nhận thấy số ether ở tài khoản thứ 3 sẽ bị giảm đi một ít. Đó là khi thực hiện giao dịch sẽ hao hụt khí gas. Bây giờ ta thực hiện gửi ether để xem số dư đã nhận được là như nào nha.

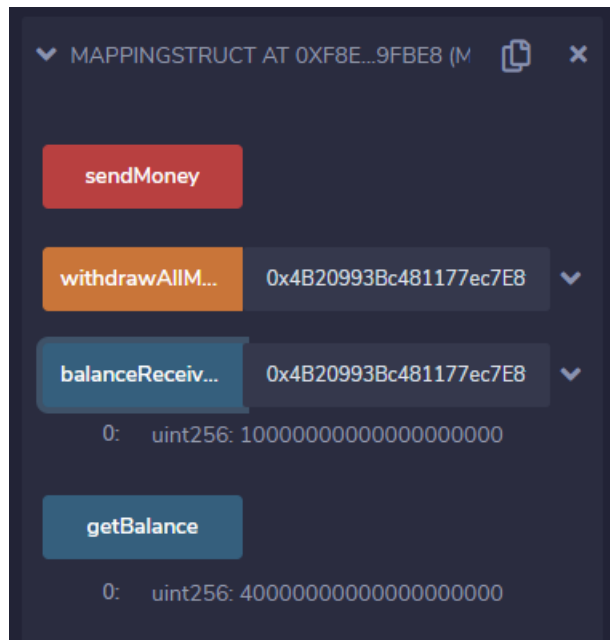
Ta nhập giá trị vào Value và kéo xuống nhấn vào sendMoney. Khi đó xem thông báo giao dịch.



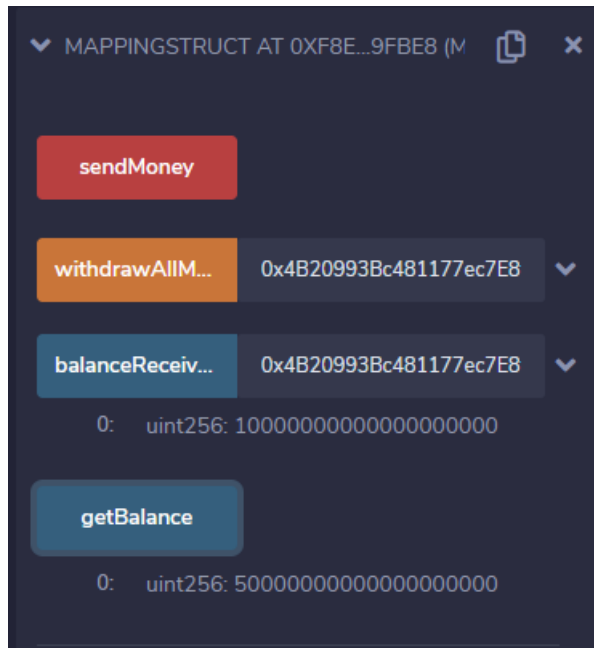
Ở đây, mình gửi đi 10 ether đó.

Và giờ thì xem số dư đã nhận được là bao nhiêu. Chúng ta nhấn vào nút balanceReceived.

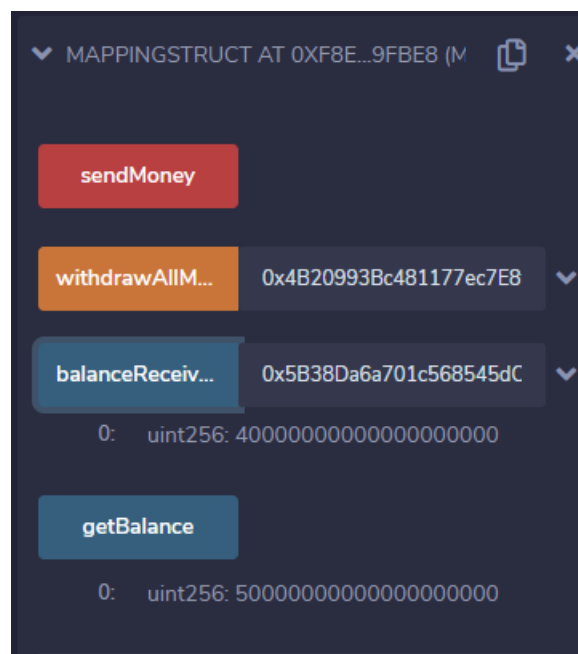
Kết quả đây nè:



Và cùng xem số dư hiện tại sẽ thay đổi như nào ta nhấn vào getBalance. Nó đã được cộng thêm 10ether.



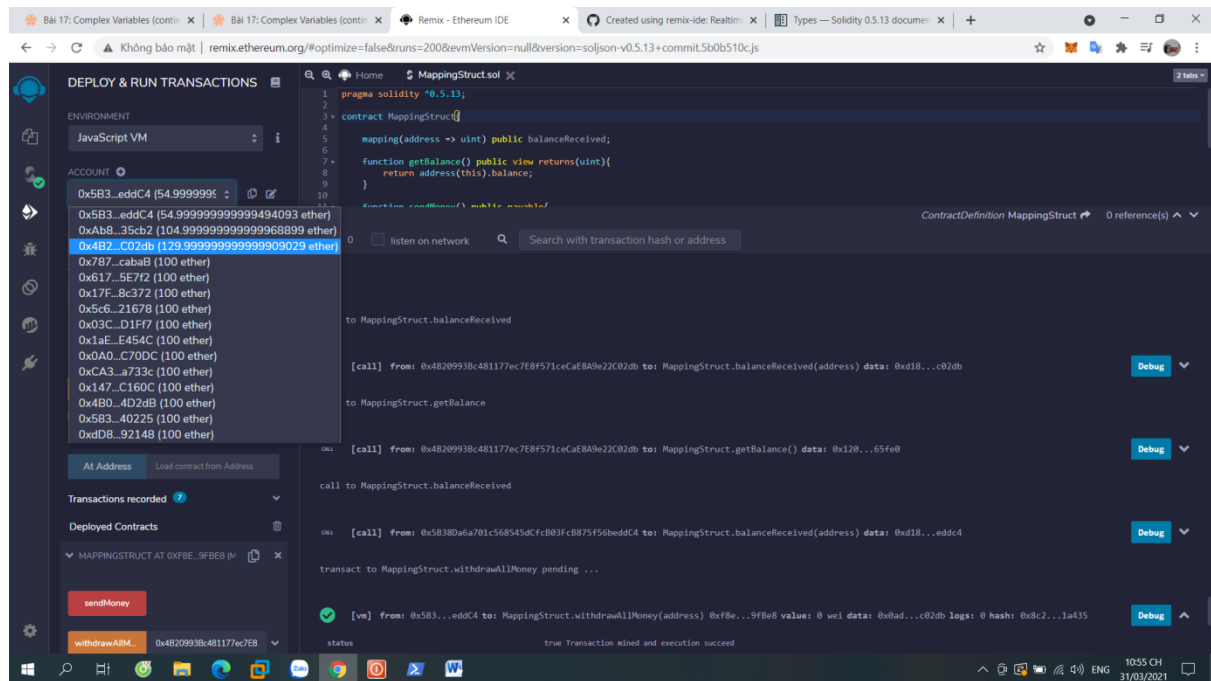
Hiện tại, Ta đang xem số dư nhận được ở địa chỉ thứ 3 và bây giờ để xem lại số dư nhận được của địa chỉ ban đầu là bao nhiêu ta copy địa chỉ ban đầu và dán vào balanceReceived.



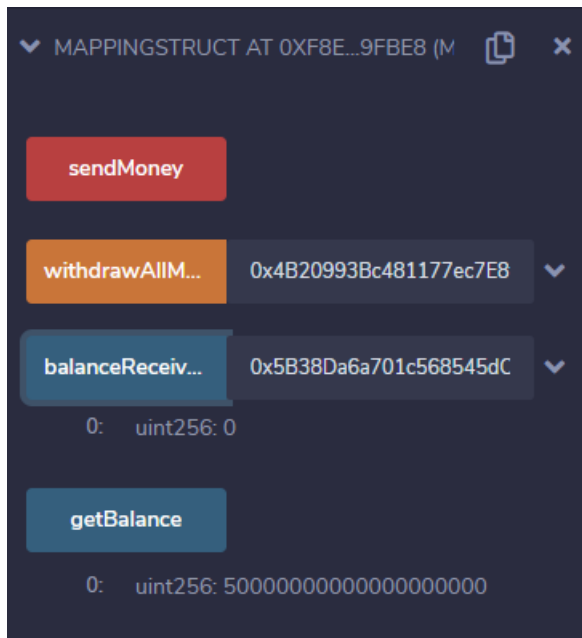
Đúng rồi chứ!

Giờ chúng ta thực hiện chuyển tổng số dư ether đã nhận từ tài khoản ban đầu sang tài khoản thứ 3 xem sao nhé. Ta chỉ cần nhấn nút withdrawAllMoney lúc này giao dịch sẽ hiển thị thông báo thành công.

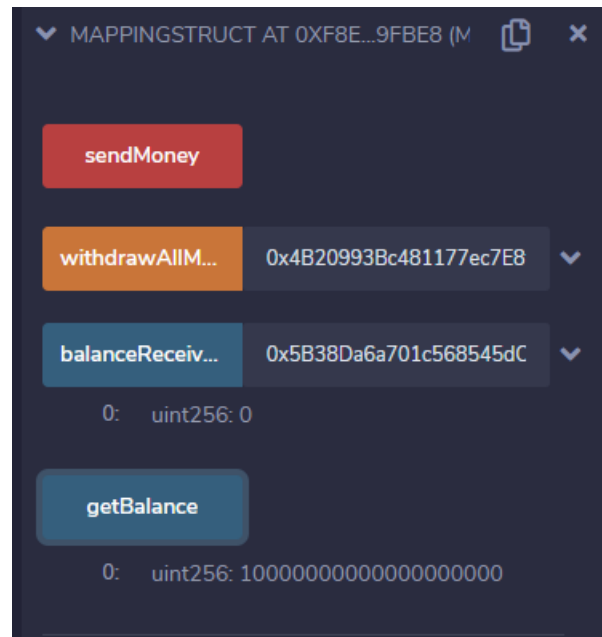
Và xem kết quả. Nhưng ở đây, vì tốn khá nhiều gas nên ta chỉ nhận được >129ether.



Lúc này, trở về lại số dư đã nhận được sẽ là 0.



Và số dư hiện tại sẽ bị trừ đi một khoảng tương ứng với số dư đã nhận được ở trên.



Chúng ta tiến hành với 1 function khác:

```
function withdrawMoney(address payable _to, uint _amount) public{
    require(balanceReceived[msg.sender] >= _amount, "not enough funds");
    balanceReceived[msg.sender] -= _amount;
    _to.transfer(_amount);
}
```

Hàm require dùng để đảm bảo điều kiện số dư nhận được của địa chỉ đó phải lớn hơn hoặc bằng số tiền chuyển đi nếu không hợp lệ sẽ thông báo lỗi “not enough funds”.

Tiến hành triển khai hợp đồng. Giao dịch sẽ triển khai và hiển thị giao dịch thành công.

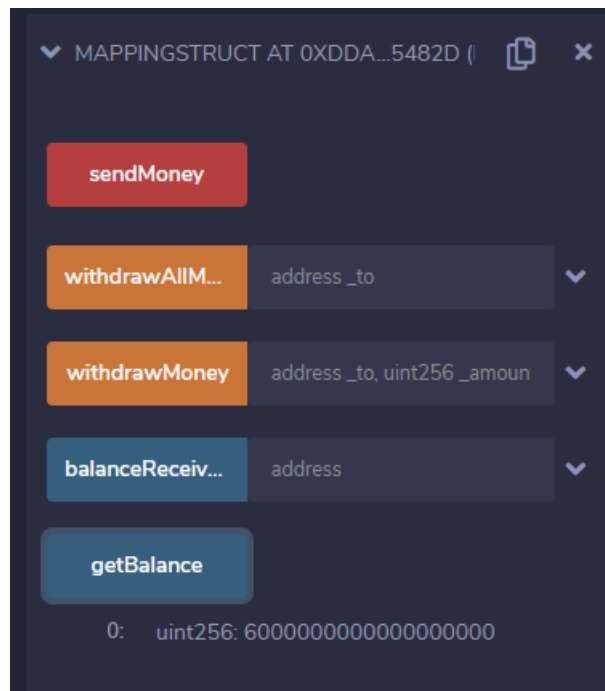
Bây giờ, ta thực hiện gửi ether. Gửi đi 6ether.

Ta kéo xuống và nhấn vào sendMoney. Giao dịch thành công.

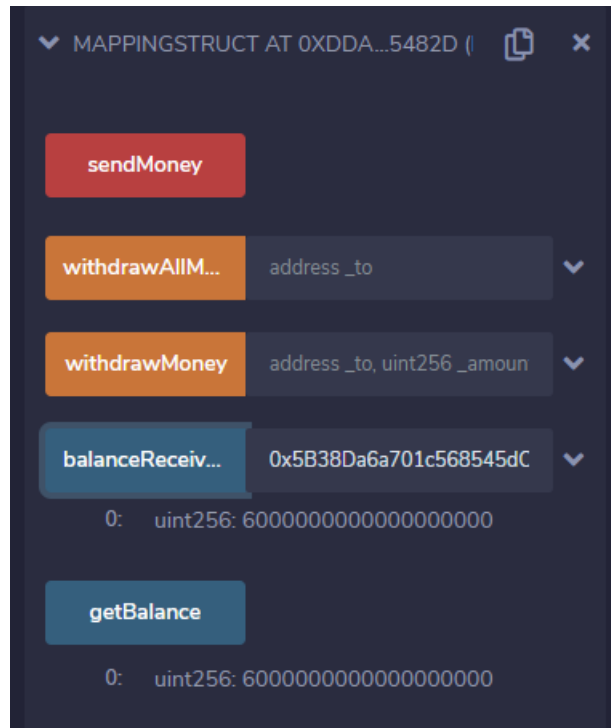
```
[vm] from: 0x583...eddC4 to: MappingStruct.sendMoney() 0xdda...5482d value: 6000000000000000 wei data: 0xcbe...dbf5a logs: 0 hash: 0x6bc...99ac8 Debug ^

status      true Transaction mined and execution succeed
transaction hash  0x6bcf2f7b3d1e7e7aa32cb02e2b5392dd30d3a6ff22d15df4097907ffe1599ac8 ⓘ
from        0x5838da6a701c568545dcfc803fc8875f5beddc4 ⓘ
to          MappingStruct.sendMoney() 0xddad340b0f1ef65169Ae5E41A8b10776a75482d ⓘ
gas         3000000 gas ⓘ
transaction cost 42335 gas ⓘ
execution cost  21063 gas ⓘ
hash         0x6bcf2f7b3d1e7e7aa32cb02e2b5392dd30d3a6ff22d15df4097907ffe1599ac8 ⓘ
input        0xcbe...dbf5a ⓘ
decoded input  {} ⓘ
decoded output {} ⓘ
logs         [] ⓘ ⓘ
value        6000000000000000 wei ⓘ
```

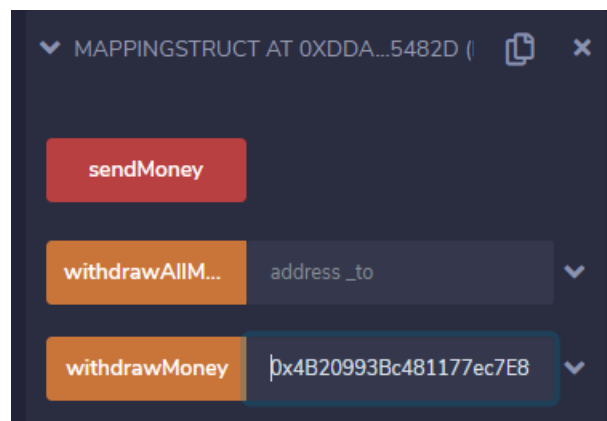
Lúc này kiểm tra số dư hiện tại.



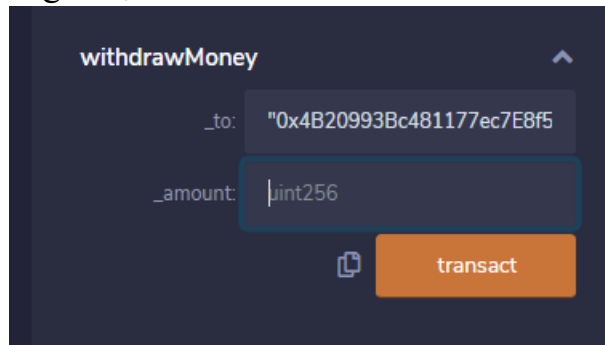
Giờ chúng ta sao chép địa chỉ và dán vào trường balanceReceived và nhấn vào balanceReceived.



Tiếp theo ta chọn địa chỉ thứ 3 và sao chép địa chỉ đó dán vào trường withdrawMoney



Ở đây ta nhập vào giá trị amount số tiền mà ta muốn chuyển tới tài khoản.



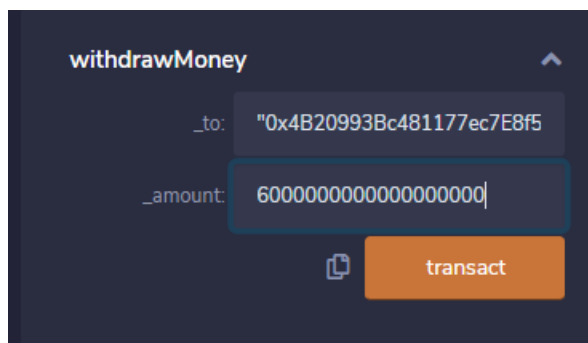
withdrawMoney

_to: "0x4B20993Bc481177ec7E8f5"

_amount: uint256

transact

Ta nhập tổng số ether đang có trong số dư nhận được luôn nha.



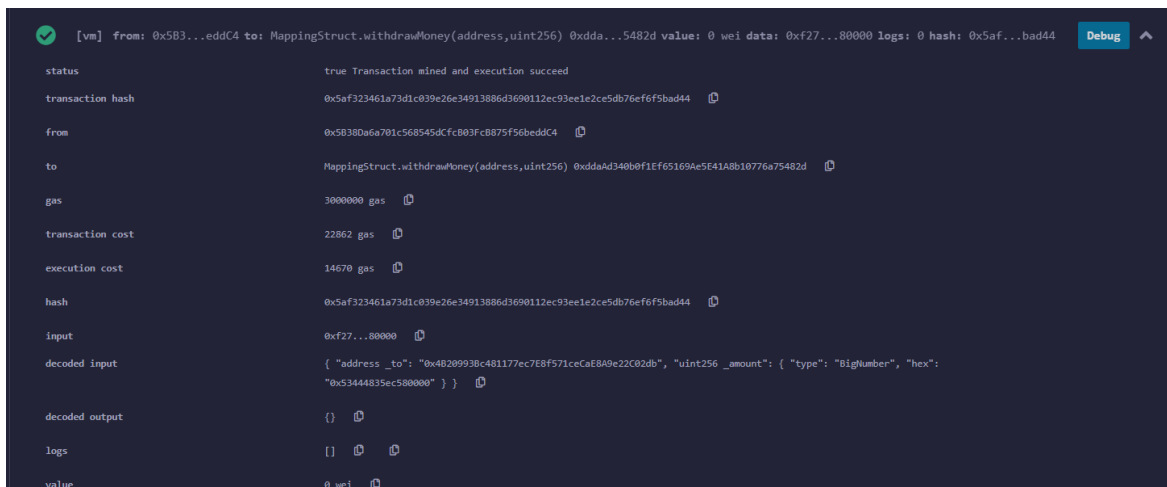
withdrawMoney

_to: "0x4B20993Bc481177ec7E8f5"

_amount: 6000000000000000000

transact

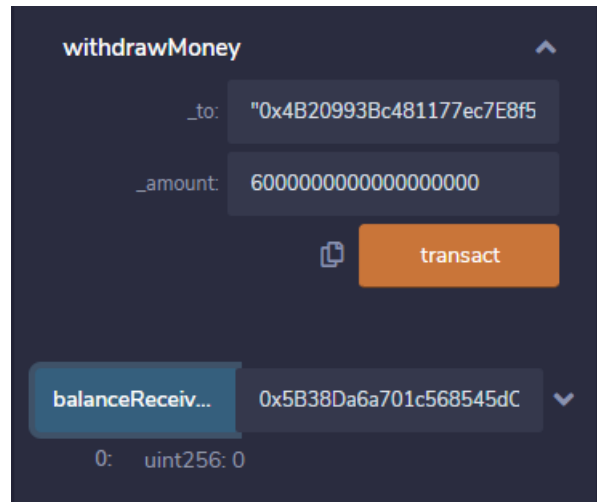
Và bắt đầu thực hiện giao dịch, giao dịch thực hiện thành công.



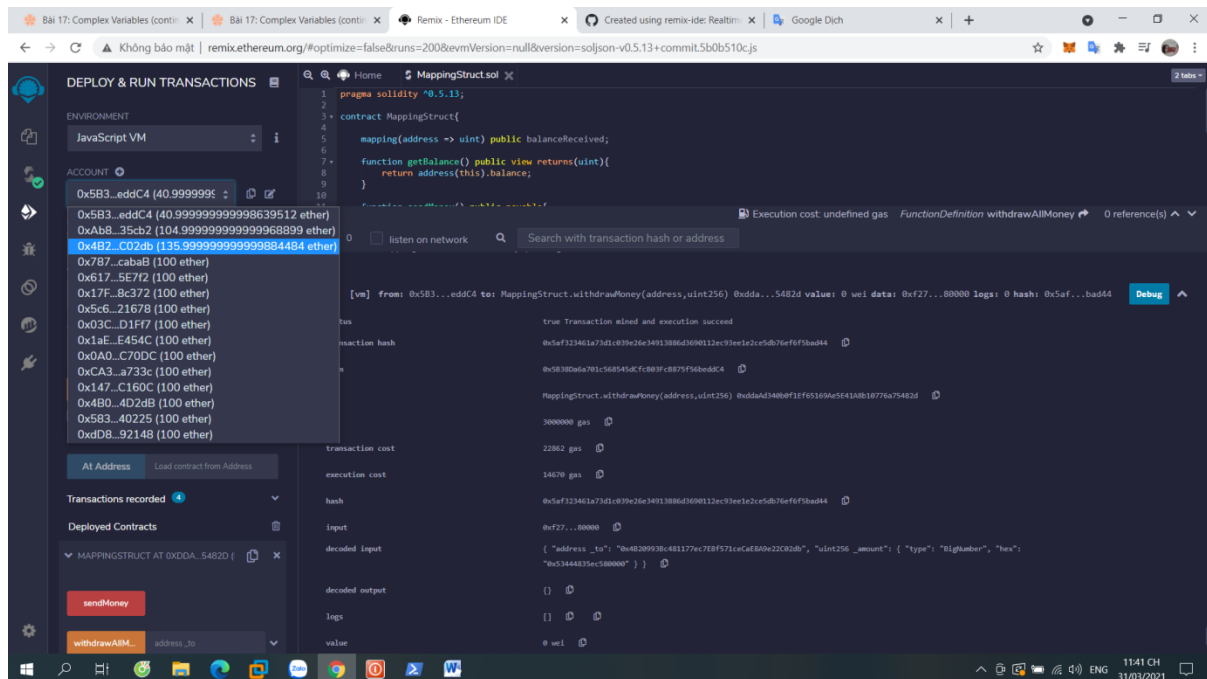
```
[vm] from: 0x5B3...eddC4 to: MappingStruct.withdrawMoney(address,uint256) 0xdda...5482d value: 0 wei data: 0xf27...80000 logs: 0 hash: 0x5af...bad44 Debug ^

status true Transaction mined and execution succeed
transaction hash 0x5af323461a73d1c039e26e34913886d3690112ec93ee1e2ce5db76ef6f5bad44
from 0x5B380a6a701c568545dCfc803fc8875f56beddC4
to MappingStruct.withdrawMoney(address,uint256) 0xddaAd340b0f1Ef65169Ae5E41A8b10776a75482d
gas 3000000 gas
transaction cost 22862 gas
execution cost 14670 gas
hash 0x5af323461a73d1c039e26e34913886d3690112ec93ee1e2ce5db76ef6f5bad44
input 0xf27...80000
decoded input { "address _to": "0x4B20993Bc481177ec7E8f571ceCaE8A9e22C02db", "uint256 _amount": { "type": "BigNumber", "hex": "0x53444835ec580000" } }
decoded output {}
logs []
value 0 wei
```

Như vậy, khi ta kiểm tra số dư được nhận số ether đã được chuyển đi và sẽ trở về 0.



Và địa chỉ được chuyển ether đến sẽ tăng lên 6ether.



Khi đó, số dư hiện tại của chúng ta cũng sẽ trở về 0.

▼

MAPPINGSTRUCT AT 0XDDA...5482D (

✖

sendMoney

withdrawAllM...

address _to

▼

withdrawMoney

▲

_to:

"0x4B20993Bc481177ec7E8f5"

_amount:

6000000000000000000

✏

transact

balanceReceiv...

0x5B38Da6a701c568545dC

▼

0: uint256: 0

getBalance

0: uint256: 0

LAB 10:

Sau khi thực hiện xong Lab 9, ta từ đó update lên và tạo 2 Struct chứa nhiều kiểu dữ liệu và nhiều thuộc tính hơn.

```
1  pragma solidity ^0.5.13;
2
3  contract MappingStruct{
4
5      struct Payment{
6          uint amount;
7          uint timestamps;
8      }
9
10     struct Balance{
11         uint totalBalance;
12         uint numPayments;
13         mapping(uint => Payment) payments;
14     }
15
16
17     mapping(address => Balance) public balanceReceived;
18
19     function getBalance() public view returns(uint){
20         return address(this).balance;
21     }
22
23     function sendMoney() public payable{
24         balanceReceived[msg.sender].totalBalance += msg.value;
25
26         Payment memory payment = Payment(msg.value, now);
27         balanceReceived[msg.sender].payments[balanceReceived[msg.sender].numPayments] = payment;
28         balanceReceived[msg.sender].numPayments++;
29     }
30
31
32     function withdrawMoney(address payable _to, uint _amount) public{
33         require(balanceReceived[msg.sender].totalBalance >= _amount, "not enough funds");
34         balanceReceived[msg.sender].totalBalance -= _amount;
35         _to.transfer(_amount);
36     }
37
38     function withdrawAllMoney(address payable _to) public {
39         uint balanceToSend = balanceReceived[msg.sender].totalBalance;
40         balanceReceived[msg.sender].totalBalance = 0;
41         _to.transfer(balanceToSend);
42     }
43
44 }
```

Cùng tìm hiểu mỗi dòng code có chức năng như thế nào nha.

Ở dòng code thứ 5, chúng ta tạo một struct Payment lưu trữ 2 thuộc tính amount và timestamps có cùng kiểu dữ liệu uint.

Ở dòng thứ 10, ta tạo một struct Balance lưu trữ 3 giá trị totalBalance, numPayments và 1 mapping. Vì bản thân kiểu struct có thể chứa các mapping và array.

Dòng thứ 17, ta tạo một ánh xạ, loại khóa ta sử dụng là address và loại giá trị là struct Balance chúng ta vừa tạo ở trên. Sau đó chúng ta đề cập ánh xạ này đến balanceReceived.

Điều này cho chúng ta tra cứu một số dữ cụ thể từ một địa chỉ và truy xuất các thuộc tính có trong struct Balance.

Trong function `sendMoney`, để có thể truy cập đến 1 thuộc tính nào đó ta cần thực hiện dấu `(.)` thuộc tính cần truy cập.

+ Một biến lưu trên memory chỉ là tạm thời và sau đó sẽ bị xóa.

Dòng code 27, dòng này sẽ lưu trữ một tham chiếu cấu trúc thanh toán trong ánh xạ `balanceReceived`.

Nhận struct `Balance` tương ứng với địa chỉ gửi hiện tại. Trong struct `Balance` này, lưu struct `payment` đã được tạo ở dòng trước vào ánh xạ thanh toán của `payments` này tại `numPayments`.

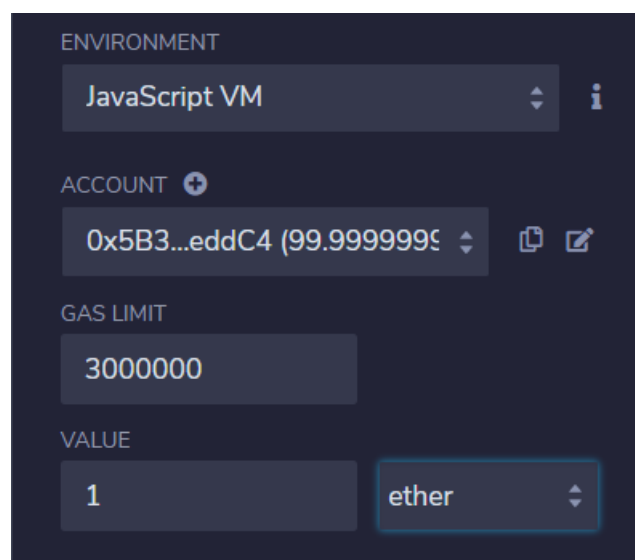
Về cơ bản `numPayments` trở thành một id duy nhất đại diện cho số nhận chúng ta nhận được số dư.

Cứ mỗi lần chúng ta gửi ether và gọi `balanceReceived` thì số lần `numPayments` sẽ tăng dần thông qua biến đếm `numPayments++`.

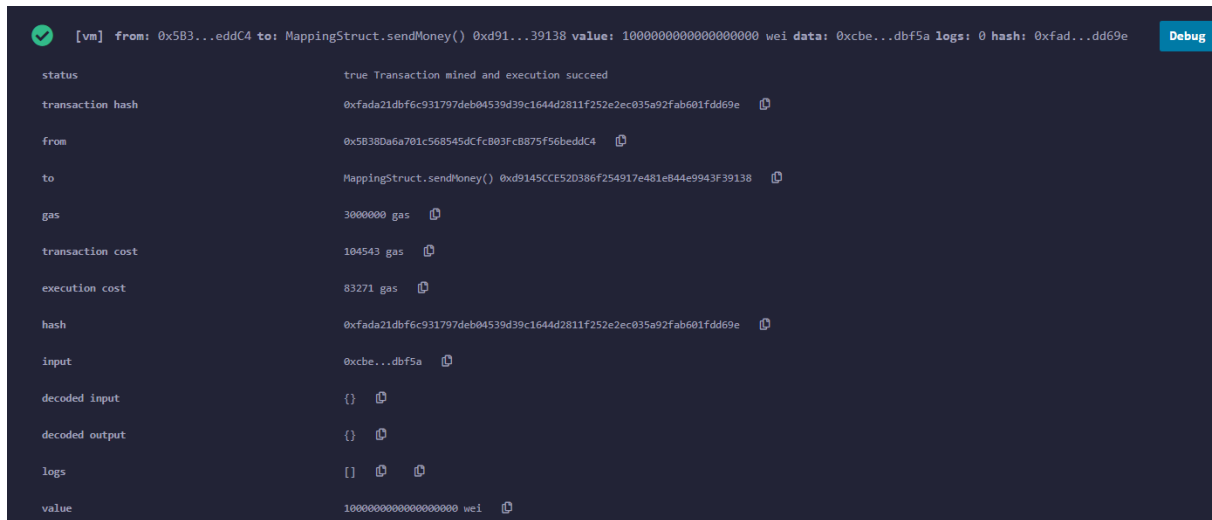
Với 2 function còn lại, số dư nhận được của địa chỉ đó sẽ truy cập đến tổng số dư. Như vậy, để các thuộc tính của struct có thể ghi vào trạng thái ta cần tham chiếu chúng.

Bây giờ, ta tiến hành triển khai contract.

Tiếp theo, ta thực hiện gửi 1 ether đến hợp đồng.



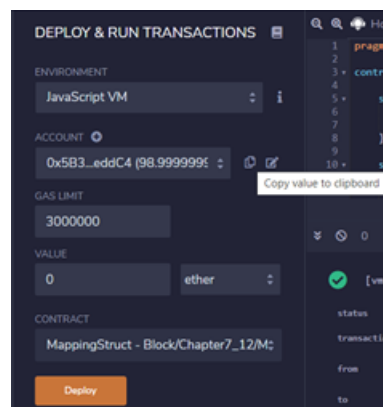
Kéo xuống và nhấp vào sendMoney. Giao dịch thực thi thành công.



The screenshot shows a transaction debug console with a green checkmark icon and a 'Debug' button. The transaction details are as follows:

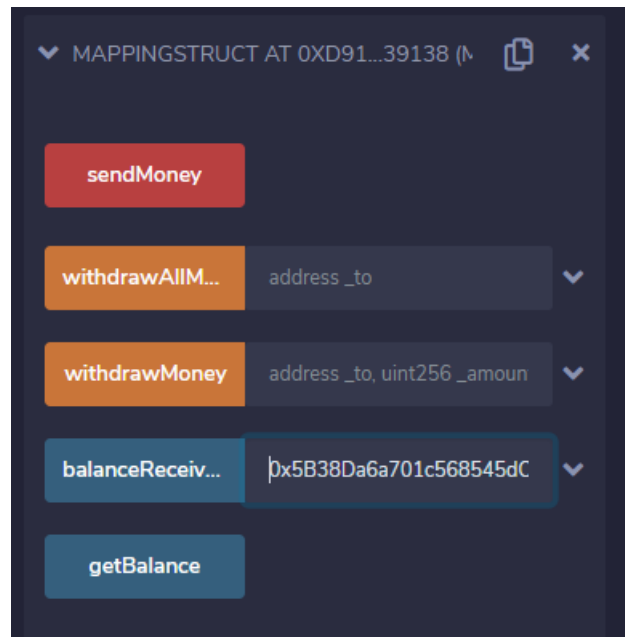
Field	Value
status	true Transaction mined and execution succeed
transaction hash	0xfada21dbf6c931797deb04539d39c1644d2811f252e2ec035a92fab601fdd69e
from	0x583806a701c568545dcfc803fc8875f56beddC4
to	MappingStruct.sendMoney() 0xd9145CC52D386f254917e481e844e9943f39138
gas	3000000 gas
transaction cost	104543 gas
execution cost	83271 gas
hash	0xfada21dbf6c931797deb04539d39c1644d2811f252e2ec035a92fab601fdd69e
input	0xcbe...dbf5a
decoded input	{}
decoded output	{}
logs	[]
value	1000000000000000000 wei

Giờ ta sao chép địa chỉ đã thực hiện gửi ether.



The screenshot shows the 'DEPLOY & RUN TRANSACTIONS' window in a development environment. The 'ENVIRONMENT' is set to 'JavaScript VM'. The 'ACCOUNT' is '0x583...eddC4 (98.999999%)'. The 'GAS LIMIT' is '3000000'. The 'VALUE' is '0' and the unit is 'ether'. The 'CONTRACT' is 'MappingStruct - Block/Chapter7_12/M;'. A 'Deploy' button is visible. On the right, a snippet of code is shown with a 'Copy value to clipboard' tooltip. Below the code, a green checkmark icon and a '[vm]' label are visible, along with a 'status' field.

Sau đó, dán vào trường balanceReceived



Giờ để xem số dư nhận được là nhiêu thì ta nhấn vào nút balanceReceived. Và số lần thực hiện là 1. Để xem sự thay đổi ta cứ thực hiện tương tự như vậy.

